ساخت دانلودر یوتیوب با پایتون

دانلود تک‌ویدیو و پلی‌لیست کامل با yt-dlp

یک پروژهٔ عملی و گام‌به‌گام پایتون

از یک پوشهٔ خالی شروع کنید و با برنامه‌ای کوچک تمام کنید که یک ویدیو، یا یک پلی‌لیست کامل را در پوشه‌ای به نام همان پلی‌لیست و با عنوان دقیق ویدیوها ذخیره می‌کند.

Alireza Khajehvandi

LinkedIn • GitHub • وب‌سایت

Instagram • Telegram • YouTube

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

روش استفاده از این آموزش

این راهنما کاملاً عملی و برای تازه‌کارهای پایتون نوشته شده است. فصل‌ها را به‌ترتیب انجام دهید و در هر **ایستگاه بررسی** توقف کنید. دستورها آمادهٔ کپی هستند؛ بااین‌حال توضیح کوتاه کنار هر دستور را بخوانید تا بدانید چه کاری انجام می‌دهد.

- فصل‌های ۱ و ۲ هدف پروژه را توضیح می‌دهند و رایانهٔ شما را آماده می‌کنند.
- فصل‌های ۳ و ۴ همهٔ ابزارهای پایتون و همهٔ تنظیم‌های yt-dlp را که در برنامه به کار رفته‌اند، هر کدام با یک مثال کوچک و قابل‌اجرا آموزش می‌دهند.
- فصل ۵ فایل downloader.py را گام‌به‌گام می‌سازد تا کامل شود.
- فصل ۶ برنامه را برای ویدیو، پلی‌لیست و صدا اجرا و آزمون می‌کند.
- فصل ۷ نشان می‌دهد استفادهٔ واقعی چه مشکل‌هایی را آشکار کرد، علت آن‌ها چگونه پیدا شد و چرا هر اصلاح درست کار می‌کند.
- فصل ۸ مشکل‌های رایج را برطرف می‌کند و ایده‌هایی برای تمرین پیشنهاد می‌دهد.

نسخه‌های فارسی و انگلیسی، کد، دستورها، نتیجه‌های مورد انتظار و نکته‌های یکسانی دارند. فقط زبان و جهت صفحه‌های آن‌ها متفاوت است. پروژهٔ کامل و فایل PDF هر دو نسخه در GitHub منتشر شده است.

نکتهٔ مهم

فقط ویدیوهایی را دانلود کنید که مالک آن‌ها هستید، مجوز انتشارشان اجازهٔ دانلود می‌دهد یا برای ذخیره‌شان اجازه گرفته‌اید. قوانین استفاده از یوتیوب و حق نشر سازندگان را رعایت کنید. این آموزش برای یادگیری پایتون است و مسئولیت نحوهٔ استفاده از برنامه با خود شماست.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فهرست مطالب

۱ چه چیزی می‌سازیم ۱

۱.۱ رفتار برنامه ۱

۲.۱ برنامه چگونه کار می‌کند ۲

۳.۱ ساختار پروژه ۲

۴.۱ چه چیزهایی لازم دارید ۲

۲ آماده‌سازی پایتون و پروژه ۴

۱.۲ بررسی پایتون ۴

۲.۲ نصب FFmpeg ۴

۳.۲ تهیهٔ پوشهٔ پروژه ۵

۴.۲ ساخت محیط جداگانه ۵

۵.۲ نوشتن فایل نیازمندی‌ها ۶

۶.۲ نصب بسته‌ها ۶

۳ ابزارهای پایتونی که به کار می‌بریم ۸

۱.۳ ماژول‌ها و import ۸

۲.۳ متغیر، رشته و in ۸

۳.۳ تابع، return و مقدار پیش‌فرض ۹

۴.۳ تصمیم‌گیری با if و else ۹

۵.۳ لیست، تاپل و دیکشنری ۹

۶.۳ مسیرها با pathlib ۱۰

۷.۳ یافتن یک برنامه با shutil.which ۱۰

۸.۳ دستور with ۱۰

۹.۳ آرگومان‌های خط فرمان با argparse ۱۱

۱۰.۳ پرسیدن از کاربر با input ۱۱

۱۱.۳ ایجاد خطای دلخواه با raise ۱۲

۱۲.۳ اجرای برنامهٔ دیگر با subprocess.run ۱۲

۱۳.۳ مدیریت خطا با try و except ۱۲

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۱۴.۳ جایگزینی متن با replace ۱۳
۱۵.۳ خط if __name__ == "__main__" ۱۳

۴ آشنایی با yt-dlp ۱۴
۱.۴ نگاه اول از داخل ترمینال ۱۴
۲.۴ به‌کاربردن yt-dlp در پایتون ۱۴
۳.۴ تنظیم‌هایی که برنامهٔ ما به کار می‌برد ۱۵
۱.۳.۴ الگوی خروجی: نام‌گذاری فایل‌ها ۱۵
۲.۳.۴ انتخاب قالب: تعیین کیفیت ۱۵
۳.۳.۴ انتخاب کیفیت ۱۶
۴.۳.۴ ترجیح قالب‌هایی که همه‌جا پخش می‌شوند ۱۶
۵.۳.۴ فقط صدا: پس‌پردازنده‌ها ۱۶
۶.۳.۴ کوکی‌ها: وقتی یوتیوب از شما می‌خواهد وارد شوید ۱۷

۵ نوشتن گام‌به‌گام برنامه ۱۸
۱.۵ گام ۱: توصیف فایل ۱۸
۲.۵ گام ۲: واردکردن ابزارها و تعیین مقدارهای پیش‌فرض ۱۸
۳.۵ گام ۳: تشخیص پلی‌لیست ۱۹
۴.۵ گام ۴: انتخاب نام فایل‌ها ۱۹
۵.۵ گام ۵: انتخاب کیفیت ۲۰
۶.۵ گام ۶: جمع‌کردن همهٔ تنظیم‌ها ۲۱
۷.۵ گام ۷: بررسی اینکه FFmpeg واقعاً کار می‌کند ۲۲
۸.۵ گام ۸: دانلود ۲۳
۹.۵ گام ۹: خواندن خط فرمان ۲۴
۱۰.۵ گام ۱۰: شروع برنامه ۲۵
۱۱.۵ برنامهٔ کامل ۲۵

۶ اجرا و آزمون برنامه ۳۰
۱.۶ نمایش صفحهٔ راهنما ۳۰
۲.۶ دانلود یک ویدیو ۳۱
۳.۶ دانلود یک پلی‌لیست کامل ۳۲
۴.۶ ذخیرهٔ فقط صدا ۳۲
۵.۶ انتخاب کیفیت ویدیو ۳۲
۶.۶ استفاده از یک FFmpeg مشخص ۳۳
۷.۶ انتخاب پوشهٔ ذخیره ۳۳
۸.۶ روش‌های دیگر اجرا ۳۴

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۹.۶ توقف و ادامه در زمانی دیگر . . . ۳۴
۱۰.۶ آزمون‌های آفلاین . . . ۳۴

۷ درس‌هایی از استفادهٔ واقعی ۳۸
۱.۷ روشی ساده برای هر باگ . . . ۳۸
۲.۷ درس ۱: قطع شبکه نیمی از ویدیو را جا گذاشت . . . ۳۸
۱.۲.۷ نشانه . . . ۳۸
۲.۲.۷ خواندن شواهد . . . ۳۹
۳.۲.۷ پیدا کردن علت . . . ۳۹
۴.۲.۷ اصلاح و آزمون . . . ۳۹
۳.۷ درس ۲: ترمینال پیوند را تغییر داد . . . ۴۰
۱.۳.۷ نشانه . . . ۴۰
۲.۳.۷ چرا مهم است . . . ۴۰
۳.۳.۷ اصلاح . . . ۴۰
۴.۷ درس ۳: دانلودها را از کد و از گیت جدا نگه دارید . . . ۴۰
۵.۷ درس ۴: یک قابلیت تازه، فقط صدا . . . ۴۱
۶.۷ درس ۵: توقف مؤدبانه . . . ۴۱
۷.۷ درس ۶: انتخاب کیفیت و ویدیوی عمودی . . . ۴۱
۱.۷.۷ نیاز و نسخهٔ نخست . . . ۴۲
۲.۷.۷ نشانه . . . ۴۲
۳.۷.۷ اصلاح . . . ۴۲
۸.۷ درس ۷: پیدا شدن FFmpeg با کارکردن آن یکی نیست . . . ۴۲
۹.۷ درس ۸: توضیح کد باید راست بگوید . . . ۴۳
۱۰.۷ خلاصهٔ تغییرها . . . ۴۳

۸ رفع اشکال و گام‌های بعدی ۴۵
۱.۸ مشکل‌های رایج . . . ۴۵
۲.۸ تمیز نگه‌داشتن پروژه در گیت . . . ۴۷
۳.۸ ایده‌هایی برای تمرین . . . ۴۷

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۱

چه چیزی می‌سازیم

هدف فصل

بفهمید برنامهٔ نهایی چه کاری انجام می‌دهد، در یک نگاه چگونه کار می‌کند و پروژه چه فایل‌هایی دارد.

یک برنامهٔ کوتاه پایتون به نام `downloader.py` می‌نویسیم. پیوند یک ویدیوی یوتیوب را در ترمینال به آن می‌دهید و برنامه ویدیو را روی رایانهٔ شما ذخیره می‌کند.

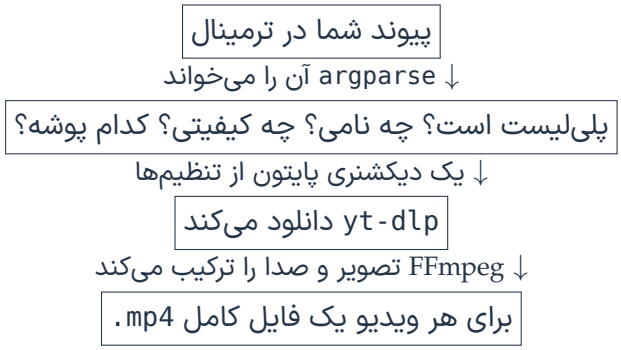
۱.۱ رفتار برنامه

- **پیوند یک ویدیو:** برنامه همان ویدیو را دانلود می‌کند و نام فایل را دقیقاً عنوان آن در یوتیوب می‌گذارد؛ مثلاً `Me at the zoo.mp4`.
- **پیوند یک پلی‌لیست:** برنامه پوشه‌ای به نام پلی‌لیست می‌سازد و همهٔ ویدیوها را در آن دانلود می‌کند. هر فایل عنوان دقیق خودش را نگه می‌دارد و یک شماره هم می‌گیرد تا ترتیب پلی‌لیست حفظ شود: `01 - First video.mp4`، `02 - Second video.mp4` و به همین ترتیب.
- **پوشهٔ ذخیره:** با گزینهٔ `-o FOLDER` محل ذخیره را انتخاب می‌کنید. اگر آن را ننویسید، فایل‌ها در پوشهٔ `downloads` کنار `downloader.py` ذخیره می‌شوند؛ فرقی نمی‌کند برنامه را از کجا اجرا کرده باشید. گیت این پوشه را نادیده می‌گیرد؛ پس ویدیوها هرگز وارد مخزن شما نمی‌شوند.
- **فقط صدا:** با گزینهٔ `--audio` برنامه فقط صدا را به شکل فایل `.mp3` ذخیره می‌کند؛ مناسب برای موسیقی، پادکست و درس‌های زبان.
- **کیفیت به انتخاب شما:** به‌طور پیش‌فرض کیفیت 720p یا نزدیک‌ترین کیفیت پایین‌تر ذخیره می‌شود. `--quality 480` یا `--quality 1080` کیفیت دیگری انتخاب می‌کند و `--exact-quality` فقط همان کیفیت دقیق را می‌پذیرد. تصویر و صدا در یک فایل `.mp4` ترکیب می‌شوند که روی گوشی، تلویزیون و همهٔ پخش‌کننده‌های رایج پخش می‌شود.
- **FFmpeg در هر مسیری:** اگر FFmpeg خودکار پیدا نشود یا نسخهٔ پیداشده خراب باشد، `--ffmpeg-location` به یک نسخهٔ سالم اشاره می‌کند. برنامه پیش از دانلود FFmpeg را بررسی می‌کند و اگر اجرا نشود، دلیلش را روشن توضیح می‌دهد.
- **رفتار دوستانه و مقاوم:** اگر پیوند را فراموش کنید، برنامه آن را از شما می‌پرسد. اگر یکی از ویدیوهای پلی‌لیست خصوصی یا حذف‌شده باشد، از آن می‌گذرد و بقیه را ادامه می‌دهد. پس از قطع شبکه خودکار دوباره تلاش می‌کند و با Ctrl+C مؤدبانه متوقف می‌شود تا همان دستور بعداً ادامه دهد.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۲.۱ برنامه چگونه کار می‌کند

کار سخت، یعنی گفت‌وگو با یوتیوب، انتخاب قالب‌ها، دانلود تکه‌تکه و نمایش پیشرفت، را `yt-dlp` انجام می‌دهد؛ یک کتابخانهٔ متن‌باز و پرطرفدار پایتون. کار ما **آماده‌کردن تنظیم‌های درست** و سپردن پیوند به آن است.



۳.۱ ساختار پروژه

در پایان، پوشهٔ شما به این شکل است. برای اجرای برنامه فقط فایل‌های پوشهٔ `code/` لازم‌اند؛ پوشهٔ `docs/` همین آموزش را نگه می‌دارد.

```
prj-youtube-downloader/
|-- code/
|   |-- downloader.py        # the complete program (218 lines)
|   |-- requirements.txt     # the packages to install
|   |-- test_downloader.py   # 18 offline tests
|   |-- README.md            # how to run the code
|   `-- downloads/           # your videos (created automatically, not in Git)
|-- docs/
|   |-- english/             # this book (English, XeLaTeX)
|   |-- persian/             # the same book in Persian (XePersian)
|   `-- README.md            # how to build the books
|-- .gitignore               # files Git must not save (videos, ...)
|-- LICENSE
`-- README.md                # project overview
```

۴.۱ چه چیزهایی لازم دارید

- رایانه‌ای با ویندوز، مک یا لینوکس و اتصال به اینترنت.
- پایتون نسخهٔ ۳٫۱۰ یا جدیدتر (روش بررسی در فصل ۲ آمده است).
- FFmpeg؛ ابزاری رایگان برای ترکیب تصویر و صدا (در فصل ۲ نصب می‌شود).
- یک ویرایشگر متن. Visual Studio Code گزینهٔ رایگان و خوبی است.

به تجربهٔ قبلی با `yt-dlp` نیاز ندارید. همهٔ مفهوم‌های پایتونی که در برنامه به کار رفته‌اند، در فصل ۳ با مثال توضیح داده شده‌اند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

ایستگاه بررسی

می‌توانید در یک جمله بگویید با پیوند یک ویدیو چه اتفاقی می‌افتد، با پیوند یک پلی‌لیست چه اتفاقی می‌افتد و اگر پوشه‌ای انتخاب نکنید، فایل‌ها کجا ذخیره می‌شوند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۲

آماده‌سازی پایتون و پروژه

هدف فصل

پایتون را بررسی کنید، FFmpeg را نصب کنید، پوشهٔ پروژه و یک محیط جداگانهٔ پایتون بسازید و بسته‌ها را نصب کنید.

۱.۲ بررسی پایتون

یک ترمینال باز کنید: در مک و لینوکس برنامهٔ Terminal، در ویندوز برنامهٔ PowerShell یا ترمینال داخل ویرایشگر. سپس اجرا کنید:

```
python --version
```

در بعضی سیستم‌های مک یا لینوکس باید از دستور `python3` استفاده کنید. هر نتیجه‌ای از Python 3.10 به بالا، مانند Python 3.12.11، مناسب است. اگر پایتون نصب نیست، آن را از `https://www.python.org/downloads/` نصب کنید. در ویندوز هنگام نصب، گزینهٔ Add python.exe to PATH را علامت بزنید.

۲.۲ نصب FFmpeg

یوتیوب بهترین تصویر و بهترین صدا را به‌صورت دو فایل جدا نگه می‌دارد. FFmpeg آن‌ها را در یک فایل `.mp4` ترکیب می‌کند. آن را با مدیر بستهٔ سیستم خود نصب کنید:

```
sudo apt install ffmpeg        # Ubuntu / Debian Linux
brew install ffmpeg            # macOS with Homebrew
winget install Gyan.FFmpeg     # Windows PowerShell
```

ترمینال را ببندید و دوباره باز کنید؛ سپس نصب را بررسی کنید:

```
ffmpeg -version
```

خط اول باید با `ffmpeg version` شروع شود. برنامه بدون FFmpeg هم کار می‌کند، اما در آن حالت فقط می‌تواند فایل‌هایی با کیفیت پایین‌تر را ذخیره کند که از قبل تصویر و صدا را با هم دارند (دلیلش در بخش ۲.۳.۴ آمده است).

اگر FFmpeg را با بازکردن یک فایل فشرده نصب کرده‌اید (که در ویندوز رایج است) و آن را به PATH اضافه نکرده‌اید، برنامه خودش آن را پیدا نمی‌کند. لازم نیست تنظیمی در سیستم تغییر دهید: مسیر آن، یعنی خود

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فایل برنامه یا پوشه‌اش، را با `--ffmpeg-location` بدهید (فصل ۶):

```
python downloader.py "LINK" --ffmpeg-location "C:\ffmpeg\bin"
```

۳.۲ تهیهٔ پوشهٔ پروژه

یکی از دو روش زیر را انتخاب کنید.

روش الف: همه‌چیز را خودتان بسازید (پیشنهادی برای یادگیری)

یک پوشهٔ خالی برای پروژه و یک پوشهٔ `code` در داخل آن بسازید و وارد آن شوید:

```
mkdir prj-youtube-downloader
cd prj-youtube-downloader
mkdir code
cd code
```

همهٔ فایل‌ها را هنگام خواندن فصل‌های ۲، ۵ و ۶ در همین پوشه تایپ خواهید کرد.

روش ب: پروژهٔ کامل را دریافت کنید

```
git clone https://github.com/alirezaaies/prj-youtube-downloader.git
cd prj-youtube-downloader/code
```

اگر به‌جای آن فایل فشرده را از گیت‌هاب دانلود کرده‌اید، آن را استخراج کنید و ترمینال را در پوشهٔ `code` آن باز کنید.

۴.۲ ساخت محیط جداگانه

محیط مجازی جعبه‌ای خصوصی از بسته‌ها برای یک پروژه است. این محیط بسته‌های این پروژه را از پروژه‌های دیگر پایتون جدا نگه می‌دارد تا هرگز با هم تداخل نکنند. **یکی** از روش‌های زیر را به کار ببرید.

محیط داخلی پایتون (venv) در مک یا لینوکس:

```
python3 -m venv .venv
source .venv/bin/activate
```

محیط داخلی پایتون (venv) در پاورشل ویندوز:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

کوندا (Anaconda یا Miniconda) در هر سیستمی:

```
conda create -n ytdl python12.3= -y
conda activate ytdl
```

اکنون ترمینال پیش از نشانگر خود عبارت `(.venv)` یا `(ytdl)` را نشان می‌دهد. هر بار که ترمینال تازه‌ای برای این پروژه باز می‌کنید، محیط را دوباره فعال کنید.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

نکتهٔ مهم

اگر پاورشل اجازهٔ اجرای `Activate.ps1` را نداد، یک بار دستور `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` را اجرا کنید، پاسخ `Y` بدهید و دوباره امتحان کنید.

۵.۲ نوشتن فایل نیازمندی‌ها

فایل `requirements.txt` بسته‌های لازم یک پروژه را فهرست می‌کند تا هر کسی بتواند همهٔ آن‌ها را با یک دستور نصب کند. در روش الف، فایل `requirements.txt` را در پوشهٔ `code` با محتوای زیر بسازید (در روش ب این فایل از قبل وجود دارد):

Listing ۱.۲: code/requirements.txt

```
# yt-dlp talks to YouTube; [default] adds its recommended helpers.
# YouTube changes often, so keep yt-dlp up to date (pip install -U ...).
yt-dlp[default]19.8.2026=<
# JavaScript runtime that yt-dlp needs to read YouTube's player.
deno0.2=<
```

- خط‌هایی که با # شروع می‌شوند، توضیح برای انسان هستند و pip آن‌ها را نادیده می‌گیرد.
- `yt-dlp[default]` بستهٔ `yt-dlp` را همراه با ابزارهای کمکی پیشنهادی‌اش نصب می‌کند. بخش داخل کروشه **افزونه** (extra) نام دارد.
- `2026.8.19=>` یعنی «این نسخه یا هر نسخهٔ جدیدتر». نسخه‌های جدیدتر را عمداً مجاز گذاشته‌ایم؛ یوتیوب مرتب تغییر می‌کند و `yt-dlp` هم برای هماهنگی با آن به‌روز می‌شود.
- `deno` یک موتور کوچک جاوااسکریپت است. پخش‌کنندهٔ وب یوتیوب معماهایی به زبان جاوااسکریپت دارد و `yt-dlp` برای حل آن‌ها از `deno` کمک می‌گیرد. بدون آن، بعضی قالب‌ها در دسترس نیستند یا دانلود شکست می‌خورد.

۶.۲ نصب بسته‌ها

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

خط اول pip، یعنی نصب‌کنندهٔ بسته‌های پایتون، را به‌روز می‌کند. خط دوم فایل `requirements.txt` را می‌خواند (`-r`) و همهٔ بسته‌های فهرست‌شده در آن را نصب می‌کند. نوشتن `python -m pip` به‌جای `pip` تضمین می‌کند که pip همان محیط **فعال** به کار برود.

به یک تفاوت کوچک در نام‌ها دقت کنید: بسته با نام `yt-dlp` (با خط تیره) نصب می‌شود، اما در پایتون با نام `yt_dlp` (با زیرخط) وارد می‌شود؛ چون نام‌ها در پایتون نمی‌توانند خط تیره داشته باشند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

ایستگاه بررسی

سه دستور زیر را اجرا کنید. هر کدام یک شمارهٔ نسخه چاپ می‌کند. اگر هر سه کار کردند، محیط شما آماده است.

```
python -c "import yt_dlp; print(yt_dlp.version.__version__)"
deno --version
ffmpeg -version
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۳

ابزارهای پایتونی که به کار می‌بریم

هدف فصل

همهٔ مفهوم‌های پایتونی موجود در `downloader.py` را، هر کدام با یک مثال کوچک که خودتان می‌توانید اجرا کنید، یاد بگیرید.

همهٔ مثال‌ها را می‌توانید در **پوستهٔ تعاملی پایتون** امتحان کنید. در ترمینالِ فعال‌شده دستور `python` را بنویسید؛ نشانگر به `>>>` تغییر می‌کند. یک خط بنویسید و Enter را بزنید تا نتیجه‌اش را ببینید. با `exit()` از پوسته خارج شوید. در مثال‌ها، خط‌هایی که با `#` شروع می‌شوند توضیح هستند و خروجی مورد انتظار در توضیحی مانند `# -> True` نشان داده شده است.

۱.۳ ماژول‌ها و import

ماژول فایلی از کدهای آمادهٔ پایتون است. `import` آن را در برنامهٔ شما در دسترس قرار می‌دهد. بعضی ماژول‌ها همراه خود پایتون هستند (**کتابخانهٔ استاندارد**) و بعضی دیگر، مانند `yt_dlp`، با `pip` نصب می‌شوند.

```
import shutil                   # use it as shutil.which(...)
from pathlib import Path        # take only Path from the pathlib module
from yt_dlp import YoutubeDL    # take only YoutubeDL from yt_dlp
```

برنامهٔ ما سه ماژول استاندارد را به کار می‌برد: `argparse` (خواندن خط فرمان)، `shutil` (یافتن برنامه‌ها) و `pathlib` (کار با پوشه‌ها). یک کتابخانهٔ نصب‌شده هم دارد: `yt_dlp`.

۲.۳ متغیر، رشته و in

متغیر نامی برای یک مقدار است. مقدارهای متنی **رشته** نام دارند و داخل گیومه نوشته می‌شوند. عملگر `in` بررسی می‌کند که آیا یک رشته داخل رشتهٔ دیگری آمده است یا نه، و نتیجهٔ `True` یا `False` می‌دهد (یک مقدار **بولی**).

```
url = "https://www.youtube.com/playlist?list=PL123"
print("list=" in url)        # -> True
print("list=" in "watch?v=abc")  # -> False
```

اف‌استرینگ (رشته‌ای که پیش از گیومه‌اش حرف `f` آمده است) مقدارها را با آکولاد داخل متن قرار می‌دهد:

```
folder = "/home/sara/Videos"
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
print(f"Saving to: {folder}")   # -> Saving to: /home/sara/Videos
```

متد `.strip()` فاصله‌ها و شکستن خط‌ها را از دو سر یک رشته حذف می‌کند. این کار برای متنی که کاربر جای‌گذاری می‌کند، مفید است:

```
print("  https://youtu.be/abc  ".strip())   # -> https://youtu.be/abc
```

۳.۳ تابع، return و مقدار پیش‌فرض

تابع بلوکی نام‌دار از کد است که می‌توانید بارها آن را صدا بزنید. `def` آن را می‌سازد و `return` نتیجه را به فراخواننده برمی‌گرداند. رشتهٔ درست زیر خط `def` یک **داک‌استرینگ** است؛ توضیحی کوتاه برای انسان. خط‌هایی که تورفتگی (چهار فاصله) دارند، به تابع تعلق دارند.

```
def greet(name, greeting="Hello"):
    """Build a short greeting."""
    return f"{greeting}, {name}!"

print(greet("Sara"))          # -> Hello, Sara!
print(greet("Sara", "Hi"))    # -> Hi, Sara!
```

`greeting="Hello"` یک **مقدار پیش‌فرض** است: وقتی فراخواننده آن آرگومان را ندهد، همین مقدار به کار می‌رود. تابع `download` ما برای پوشه و مرورگر از همین ایده استفاده می‌کند. مقدار ویژهٔ `None` یعنی «چیزی داده نشده است».

۴.۳ تصمیم‌گیری با if و else

`if` بلوک خود را فقط وقتی اجرا می‌کند که شرط درست باشد؛ در غیر این صورت `else` اجرا می‌شود. دستور `return` تابع را بی‌درنگ تمام می‌کند؛ پس کدهای بعد از آن اجرا نمی‌شوند.

```
def size_word(count):
    if count > 1:
        return "many videos"
    return "one video"

print(size_word(5))   # -> many videos
print(size_word(1))   # -> one video
```

پایتون مقدارهای خالی را هم نادرست در نظر می‌گیرد: `""`، `None` و `0` نادرست هستند و هر رشتهٔ غیرخالی درست است. پس `if browser:` یعنی «اگر نام مرورگری داده شده است». عملگر `or` نخستین مقدار درست را برمی‌گرداند:

```
print(None or "typed by user")      # -> typed by user
print("from command" or "typed")    # -> from command
```

۵.۳ لیست، تاپل و دیکشنری

- **لیست** مقدارها را به‌ترتیب نگه می‌دارد و قابل تغییر است: `["a", "b"]`.
- **تاپل** مانند لیست است، اما تغییر نمی‌کند: `("firefox",)`. تاپل یک‌عضوی به ویرگول پایانی نیاز دارد؛ `("firefox")` فقط یک رشته است.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

- **دیکشنری** جفت‌های **کلید: مقدار** را نگه می‌دارد؛ مانند فرمی کوچک با خانه‌های برچسب‌دار.

```
options = {"format": "best", "ignoreerrors": True}
print(options["format"])           # -> best
options["cookiesfrombrowser"] = ("firefox",)   # add a new key
print(len(options))                # -> 3
```

yt-dlp همهٔ تنظیم‌هایش را به شکل یک دیکشنری دریافت می‌کند؛ پس دیکشنری مهم‌ترین نوع داده در برنامهٔ ماست.

۶.۳ مسیرها با pathlib

Path مسیر یک فایل یا پوشه را نشان می‌دهد و در ویندوز، مک و لینوکس یکسان کار می‌کند.

```
from pathlib import Path

p = Path("~/Videos").expanduser()   # ~ becomes your home folder
print(p)                             # -> /home/sara/Videos
print(p / "Music")                   # -> /home/sara/Videos/Music
print(Path("code/downloader.py").resolve().parent)  # full folder path
p.mkdir(parents=True, exist_ok=True) # create it (and its parents) if missing
```

- __file__ متغیری ویژه است که مسیر فایل پایتونِ در حال اجرا را نگه می‌دارد.
- .resolve() مسیر را به مسیر کامل (مطلق) تبدیل می‌کند.
- .parent پوشه‌ای است که آن مسیر را در بر دارد.
- exist_ok=True یعنی «اگر پوشه از قبل وجود دارد، خطا نده».

بنابراین Path(__file__).resolve().parent یعنی «پوشه‌ای که این اسکریپت در آن است». این دقیقاً پوشهٔ پیش‌فرض ذخیره در برنامهٔ ماست.

۷.۳ یافتن یک برنامه با shutil.which

shutil.which(name) یک دستور را روی رایانهٔ شما جست‌وجو می‌کند. اگر دستور وجود داشته باشد، مسیرش را برمی‌گرداند و اگر نباشد، None:

```
import shutil
print(shutil.which("ffmpeg"))        # -> /usr/bin/ffmpeg
print(shutil.which("no-such-tool"))  # -> None
```

۸.۳ دستور with

بعضی چیزها باید باز و بعداً به‌درستی بسته شوند. with هر دو کار را انجام می‌دهد: آن را باز می‌کند، بلوک تورفته را اجرا می‌کند و در پایان می‌بندد؛ حتی اگر خطایی رخ دهد.

```
with open("note.txt", "w") as f:   # open a file for writing
    f.write("Hello")
# here the file is already closed
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

ما `with YoutubeDL(...) as ydl:` را به همین شکل به کار می‌بریم.

۹.۳ آرگومان‌های خط فرمان با argparse

`argparse` کلمه‌هایی را می‌خواند که پس از `python script.py` نوشته می‌شوند. این ماژول به‌طور خودکار صفحهٔ راهنمای `--help` می‌سازد و خطاهای روشن نشان می‌دهد. این کد را با نام `hello.py` ذخیره کنید:

```
import argparse

parser = argparse.ArgumentParser(description="Say hello.")
parser.add_argument("name", nargs="?", help="who to greet")
parser.add_argument("-t", "--times", default="1", help="how many times")
args = parser.parse_args()
print(f"Hello {args.name}! " * int(args.times))
```

```
python hello.py Sara            # -> Hello Sara!
python hello.py Sara -t 2       # -> Hello Sara! Hello Sara!
python hello.py --help          # shows the automatic help page
```

- `"name"` خط تیره ندارد؛ پس آرگومانی **موقعیتی** است و مقدارش بر اساس جایگاهش داده می‌شود.
- `nargs="?"` آن را اختیاری می‌کند؛ اگر داده نشود، مقدارش `None` است.
- `"-t", "--times"` یک **گزینه** است: نامی کوتاه و نامی بلند برای یک تنظیم، همراه با مقدار پیش‌فرض.
- پس از `parse_args()` مقدارها با `args.name` و `args.times` در دسترس هستند.

بعضی گزینه‌ها کلید ساده‌ای برای روشن یا خاموش‌کردن هستند و مقداری نمی‌گیرند. این گزینه‌ها با تنظیم `action="store_true"` ساخته می‌شوند. در این حالت اگر گزینه نوشته شود مقدارش `True` و اگر نوشته نشود `False` است.

```
parser.add_argument("-l", "--loud", action="store_true", help="shout")
# python hello.py Sara          -> args.loud is False
# python hello.py Sara --loud   -> args.loud is True
```

۱۰.۳ پرسیدن از کاربر با input

`input(message)` پیامی نشان می‌دهد، منتظر می‌ماند تا کاربر Enter را بزند و متن تایپ‌شده را به شکل رشته برمی‌گرداند:

```
link = input("Paste a link: ").strip()
print(f"You pasted {link}")
```

همهٔ چیزهایی که در ترمینال تایپ می‌شوند به شکل متن می‌رسند. با `type=int`، `argparse` مقدار را به عدد صحیح تبدیل می‌کند و اگر ممکن نباشد، با پیامی روشن متوقف می‌شود:

```
parser.add_argument("--times", type=int, default=1)
# python hello.py Sara --times 3    -> args.times is the number 3
# python hello.py Sara --times abc  -> error: invalid int value: 'abc'
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۱۱.۳ ایجاد خطای دلخواه با raise

raise یک تابع را عمداً متوقف می‌کند و خطایی گزارش می‌دهد. استثنایی را انتخاب کنید که مشکل را توصیف کند: ValueError برای مقدار نادرست و RuntimeError برای مشکلی که هنگام اجرا پیش آمده است.

```
def area(width, height):
    if width <= 0 or height <= 0:
        raise ValueError("sizes must be positive")
    return width * height
```

وقتی داخل except یک خطا را به خطای دیگری تبدیل می‌کنید، این‌گونه بنویسید:

```
raise NewError("...") from exc
```

خطای اصلی پیوست باقی می‌ماند و این به رفع اشکال کمک می‌کند.

۱۲.۳ اجرای برنامهٔ دیگر با subprocess.run

ماژول subprocess برنامه‌های دیگر را اجرا می‌کند؛ گویی آن‌ها را در ترمینال تایپ کرده‌اید. تابع run آن تا پایان کار برنامه صبر می‌کند و یک شیء نتیجه برمی‌گرداند.

```
import subprocess
result = subprocess.run(["ffmpeg", "-version"], capture_output=True,
                        text=True, timeout=15)
print(result.returncode)            # -> 0 means success
print(result.stdout.splitlines()[0])  # -> ffmpeg version 1.1.6 ...
```

- فرمان یک لیست است: نام برنامه و سپس هر آرگومان.
- capture_output=True خروجی برنامه را به‌جای چاپ، در result.stdout و result.stderr نگه می‌دارد. text=True آن را به شکل رشته برمی‌گرداند، نه بایت.
- timeout=15 اگر برنامه بیش از ۱۵ ثانیه طول بکشد، خطای TimeoutExpired ایجاد می‌کند.
- returncode **کد خروج** برنامه است: صفر یعنی موفقیت و هر عدد دیگر یعنی شکست. اگر برنامه اصلاً اجرا نشود، پایتون خطای OSError مانند FileNotFoundError ایجاد می‌کند.

۱۳.۳ مدیریت خطا با try و except

وقتی هنگام اجرای برنامه مشکلی پیش بیاید، پایتون یک **استثنا** (exception) ایجاد می‌کند و اگر کسی آن را مدیریت نکند، با یک گزارش خطای طولانی (traceback) متوقف می‌شود. try یک بلوک را اجرا می‌کند؛ اگر استثنای نام‌برده‌شده در آن رخ دهد، به‌جای توقف برنامه، بلوک except متناظر اجرا می‌شود.

```
try:
    number = int(input("Type a number: "))
except ValueError:
    print("That was not a number.")
```

زدن Ctrl+C در ترمینال، استثنای KeyboardInterrupt را ایجاد می‌کند. برنامهٔ ما آن را می‌گیرد تا به‌جای گزارش خطا، یک جملهٔ دوستانه چاپ کند. فقط استثناهایی را بگیرید که انتظارشان را دارید؛ گرفتن همهٔ خطاها، باگ‌های واقعی را هم پنهان می‌کند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۱۴.۳ جایگزینی متن با replace

`text.replace(old, new)` یک رونوشت از رشته برمی‌گرداند که در آن هر `old` به `new` تبدیل شده است. جایگزینی با رشتهٔ خالی یعنی حذف. در کد پایتون یک بک‌اسلش به شکل `"\\"` نوشته می‌شود، چون یک بک‌اسلش تنها یک **دنبالهٔ فرار** مانند `\n` (خط جدید) را شروع می‌کند.

```
link = "https://youtu.be/abc\\?si\\=xyz"
print(link)                    # -> https://youtu.be/abc\?si\=xyz
print(link.replace("\\", ""))  # -> https://youtu.be/abc?si=xyz
```

۱۵.۳ خط if __name__ == "__main__"

وقتی یک فایل را مستقیم اجرا می‌کنید، پایتون متغیر `__name__` آن را برابر `"__main__"` می‌گذارد. وقتی فایل دیگری (مثلاً فایل آزمون ما) آن را وارد می‌کند، مقدار `__name__` نام ماژول است. پس این بلوک یعنی «برنامه را فقط وقتی شروع کن که این فایل مستقیم اجرا شده باشد»:

```
if __name__ == "__main__":
    main()
```

به لطف آن، آزمون‌ها می‌توانند تابع‌های ما را وارد کنند بدون اینکه دانلودی شروع شود.

خودتان امتحان کنید

پوستهٔ پایتون را باز کنید و مثال‌های `pathlib` و `shutil.which` را روی رایانهٔ خودتان اجرا کنید. نتیجه‌ها را با توضیح‌ها مقایسه کنید.

ایستگاه بررسی

می‌توانید تابع، دیکشنری، `Path(__file__).resolve().parent`، `shutil.which`، `argparse`، `raise`، `subprocess.run` و `try/except` را هر کدام در یک جمله توضیح دهید.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۴

آشنایی با yt-dlp

هدف فصل

بفهمید `yt-dlp` چگونه از داخل پایتون به کار می‌رود و هر تنظیم در دیکشنری تنظیم‌های ما چه کاری انجام می‌دهد.

`yt-dlp` یک دانلودر رایگان و متن‌باز برای یوتیوب و بسیاری از سایت‌های دیگر است (`https://github.com/yt-dlp/yt-dlp`). آن را به دو شکل می‌توان به کار برد: به‌صورت دستور ترمینال و به‌صورت کتابخانهٔ پایتون. برنامهٔ ما از کتابخانه استفاده می‌کند، اما امتحان‌کردن دستور ترمینال در ابتدا کمک می‌کند ببینید چه کاری انجام می‌دهد.

۱.۴ نگاه اول از داخل ترمینال

```
yt-dlp --version
yt-dlp -F "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

دستور دوم چیزی دانلود نمی‌کند. گزینهٔ `-F` همهٔ **قالب‌هایی** را که یوتیوب برای آن ویدیو ارائه می‌کند، فهرست می‌کند. ردیف‌هایی فقط‌صدا، ردیف‌هایی فقط‌تصویر و چند ردیف با هر دو را خواهید دید. دقت کنید که بهترین کیفیت‌ها همیشه **فقط تصویر** یا **فقط صدا** هستند. به همین دلیل برای ترکیب آن‌ها به FFmpeg نیاز داریم.

نکتهٔ مهم

پیوند یوتیوب را در ترمینال همیشه داخل گیومه بنویسید. پیوندها اغلب نویسهٔ `&` دارند که ترمینال در غیر این صورت آن را جداکنندهٔ دستورها در نظر می‌گیرد.

۲.۴ به‌کاربردن yt-dlp در پایتون

در پایتون، `yt-dlp` فقط به سه گام نیاز دارد: یک دیکشنری تنظیم بنویسید، با آن یک شیء `YoutubeDL` بسازید و `download` را با یک **لیست** از پیوندها صدا بزنید.

```
from yt_dlp import YoutubeDL

options = {"outtmpl": "%(title)s.%(ext)s"}
with YoutubeDL(options) as ydl:
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    result = ydl.download(["https://www.youtube.com/watch?v=jNQXAC9IVRw"])
print(result)   # -> 0 means success
```

download وقتی همهٔ پیوندها موفق باشند، 0 و در صورت بروز مشکل عددی دیگر برمی‌گرداند. برنامهٔ ما از همین عدد برای چاپ پیام پایانی استفاده می‌کند.

۳.۴ تنظیم‌هایی که برنامهٔ ما به کار می‌برد

کلید	کاربرد
`format`	کدام تصویر و صدا دانلود شود.
`format_sort`	ترتیب ترجیح؛ برای نمونه، اول کیفیت.
`ffmpeg_location`	کدام FFmpeg استفاده شود، وقتی در PATH نیست.
`merge_output_format`	قالب فایل هنگام ترکیب تصویر و صدا: mp4.
`paths`	پوشهٔ پایهٔ دانلودها: {"home": folder}.
`outtmpl`	**الگوی خروجی**؛ یعنی نام‌گذاری هر فایل.
`ignoreerrors`	True: از ویدیوی ناموفق بگذر و بقیه را ادامه بده.
`retries`	پس از قطع شبکه، تا ۱۰ بار دوباره تلاش کن.
`postprocessors`	کارهایی که **پس از** دانلود انجام می‌شوند، مانند تبدیل به MP3.
`cookiesfrombrowser`	استفاده از ورود یوتیوبِ یک مرورگر: ("firefox",).

۱.۳.۴ الگوی خروجی: نام‌گذاری فایل‌ها

الگوی خروجی یک رشتهٔ معمولی است که خانه‌های ویژه‌ای به شکل %(field)s دارد. yt-dlp هر خانه را با اطلاعات ویدیو جایگزین می‌کند:

خانه	جایگزین آن
%(title)s	عنوان دقیق ویدیو
%(ext)s	پسوند فایل، مانند mp4
%(playlist_title)s	عنوان پلی‌لیست
%(playlist_index)02d	جایگاه در پلی‌لیست، به‌صورت دو رقمی

s یعنی «به شکل متن» و 02d یعنی «به شکل عدد صحیح با دست‌کم دو رقم و پرشده با صفر»؛ پس ۳ به 03 تبدیل می‌شود. این صفرها باعث می‌شوند فایل‌ها در مدیر فایل به‌درستی مرتب شوند (02 پیش از 10).

نویسهٔ / در الگو یک پوشه می‌سازد. پس برای پلی‌لیستی به نام Python Basics که ویدیوی سومش Loops نام دارد، الگوی

```
%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s
```

نتیجهٔ Python Basics/03 - Loops.mp4 را می‌سازد. اگر عنوانی نویسه‌هایی داشته باشد که در نام فایل مجاز نیستند (مانند / یا : در ویندوز)، yt-dlp خودش آن‌ها را به شکلی امن جایگزین می‌کند. عنوان‌های فارسی و دیگر زبان‌ها همان‌طور که هستند، حفظ می‌شوند.

۲.۳.۴ انتخاب قالب: تعیین کیفیت

مقدار format یک عبارت کوچک است:

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
bestvideo*+bestaudio/best
```

- `bestvideo*` بهترین جریان تصویر است؛ ستاره اجازه می‌دهد جریان‌هایی هم که اتفاقاً صدا دارند، انتخاب شوند.
- `+bestaudio` بهترین جریان صدا را اضافه می‌کند؛ این دو با FFmpeg ترکیب می‌شوند.
- `/best` گزینهٔ جایگزین است: «اگر ممکن نبود، بهترین فایل تکی را بگیر که از قبل تصویر و صدا را با هم دارد».

ترکیب‌کردن به FFmpeg نیاز دارد. بدون آن فقط `best` را درخواست می‌کنیم که هرگز به ترکیب نیاز ندارد. بهترین فایل‌های تکی یوتیوب معمولاً فقط کیفیت 360p دارند؛ پس نصب FFmpeg به‌شدت توصیه می‌شود.

۳.۳.۴ انتخاب کیفیت

کیفیت یک ویدیو به نام **ضلع کوتاه** تصویرش نامیده می‌شود. ویدیوی افقی ۱۲۸۰×۷۲۰ «720p» است؛ ویدیوی عمودی گوشی با ابعاد ۷۲۰×۱۲۸۰ هم همین‌طور. در `yt-dlp` این ضلع کوتاه `res` نام دارد. برنامه آن را به دو شکل به کار می‌برد.

حالت عادی: یک ترجیح. نخستین قانون `format_sort` عبارت `res:720` است. یعنی: بزرگ‌ترین ویدیو **تا** ۷۲۰ را ترجیح بده؛ اگر نبود، کوچک‌ترین ویدیوی بالاتر از آن را بگیر. ترجیح هرگز شکست نمی‌خورد؛ پس هر ویدیویی دانلود می‌شود، حتی ویدیوی قدیمی‌ای که فقط کیفیت 240p دارد.

حالت دقیق: یک فیلتر. کروشه‌های پس از نام قالب، فقط قالب‌هایی را نگه می‌دارند که از یک آزمون بگذرند:

```
bestvideo*[height=1080][width>=1080]     wide video, exactly 1080p
bestvideo*[width=1080][height>=1080]     vertical video, exactly 1080p
```

فیلتر **می‌تواند** شکست بخورد. اگر هیچ قالبی از آن نگذرد، `yt-dlp` پیام زیر را گزارش می‌کند و از آن ویدیو می‌گذرد:

```
Requested format is not available
```

فصل ۷ توضیح می‌دهد چرا یک فیلتر ساده مانند `[height<=720]` کافی نبود.

۴.۳.۴ ترجیح قالب‌هایی که همه‌جا پخش می‌شوند

`format_sort` همچنین تعیین می‌کند از میان چند قالب خوب کدام برنده شود. یوتیوب اغلب کدک‌های جدیدی (AV1، VP9، Opus) ارائه می‌کند که بعضی تلویزیون‌ها، گوشی‌ها و پخش‌کننده‌های قدیمی نمی‌توانند بازشان کنند. لیست

```
["res:720", "vcodec:h264", "acodec:m4a"]
```

یعنی: اول کیفیت درخواستی، سپس تصویر H.264 و سپس صدای AAC (`m4a`) را ترجیح بده. نتیجه یک فایل `.mp4` است که با کیفیت انتخابی شما همه‌جا پخش می‌شود.

۵.۳.۴ فقط صدا: پس‌پردازنده‌ها

برای فقط صدا، قالب به‌سادگی `bestaudio/best` است: بهترین جریان صدا، یا اگر صدای جداگانه‌ای نبود، بهترین فایل. بهترین صدای یوتیوب یک فایل `.m4a` (AAC) یا `.webm` (Opus) است. برای گرفتن فایل `.mp3`، که همهٔ گوشی‌ها، خودروها و پخش‌کننده‌های قدیمی آن را می‌شناسند، یک **پس‌پردازنده** درخواست می‌کنیم؛ گامی که `yt-dlp` پس از دانلود اجرا می‌کند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
"postprocessors": [
    {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
     "preferredquality": "192"},
]
```

- `postprocessors` یک **لیست** است، چون چند گام می‌توانند پشت سر هم اجرا شوند؛ هر گام یک دیکشنری است.
- `"key"` نام گام است. `FFmpegExtractAudio` از FFmpeg می‌خواهد فایل دانلودشده را به صدا تبدیل کند.
- `preferredcodec` قالب خروجی و `preferredquality` نرخ بیت را بر حسب کیلوبیت بر ثانیه تعیین می‌کند: 192 برای گفتار واضح و موسیقی خوب با حجم متوسط مناسب است (حدود ۱٫۴ مگابایت برای هر دقیقه).

پس از تبدیل، `yt-dlp` فایل اصلی `.m4a` را پاک می‌کند و فقط `.mp3` باقی می‌ماند. الگوی خروجی تغییری نمی‌کند؛ پس فایل صوتی هم همان عنوان دقیق را می‌گیرد و پلی‌لیست هم همان پوشهٔ شماره‌گذاری‌شده را.

۶.۳.۴ کوکی‌ها: وقتی یوتیوب از شما می‌خواهد وارد شوید

گاهی یوتیوب با پیام Sign in to confirm you're not a bot پاسخ می‌دهد. این اتفاق با VPN و شبکه‌های اشتراکی بیشتر رخ می‌دهد. **کوکی** دادهٔ کوچکی است که یک وب‌سایت در مرورگر شما ذخیره می‌کند تا ورود شما را به خاطر بسپارد. با `cookiesfrombrowser`، `yt-dlp` کوکی‌های یوتیوبِ یکی از مرورگرهای روی رایانهٔ خودتان را می‌خواند و درست مانند همان مرورگر از ورود شما استفاده می‌کند.

نکتهٔ مهم

کوکی‌ها به اندازهٔ گذرواژه خصوصی هستند. برنامه آن‌ها را فقط روی رایانهٔ خودتان می‌خواند و هرگز جایی ذخیره یا ارسال نمی‌کند. فایل کوکی را هرگز با کسی به اشتراک نگذارید و هرگز در گیت قرار ندهید.

ایستگاه بررسی

بدون اجرای چیزی، نام فایلی را بنویسید که الگوی زیر

```
%(playlist_index)02d - %(title)s.%(ext)s
```

برای ویدیوی هفتم با عنوان Functions و پسوند `mp4` می‌سازد. پاسخ: `07 - Functions.mp4`.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۵

نوشتن گام‌به‌گام برنامه

هدف فصل

فایل `downloader.py` را تکه‌تکه بسازید و همهٔ خط‌های آن را بفهمید. در پایان این فصل برنامه کامل است.

یک فایل خالی به نام `downloader.py` در پوشهٔ `code` بسازید و هر تکهٔ زیر را به‌ترتیب در آن تایپ کنید. شماره‌های کنار کد، شمارهٔ خط‌ها در فایل نهایی هستند؛ پس همیشه می‌بینید هر تکه کجا قرار می‌گیرد. میان تابع‌ها دو خط خالی بگذارید؛ این سبک رایج پایتون است (PEP 8). توضیح‌های داخل کد (خط‌هایی که با # شروع می‌شوند) هم کار هر خط را می‌گویند تا فایل حتی بدون این کتاب قابل‌فهم بماند.

۱.۵ گام ۱: توصیف فایل

```
"""Download one YouTube video or a whole playlist with yt-dlp.

Examples:
    python downloader.py "VIDEO_OR_PLAYLIST_LINK"
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 480
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 1080 --exact-quality
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --ffmpeg-location /opt/ffmpeg/
    bin
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
    python downloader.py          # asks for the link
"""
```

رشته‌ای که در ابتدای فایل می‌آید، **داک‌استرینگ ماژول** نام دارد. پایتون هنگام اجرا آن را نادیده می‌گیرد، اما به هر خواننده‌ای می‌گوید این فایل چه کاری انجام می‌دهد و چگونه به کار می‌رود. هر مثال یکی از گزینه‌های برنامهٔ نهایی را نشان می‌دهد.

۲.۵ گام ۲: واردکردن ابزارها و تعیین مقدارهای پیش‌فرض

```
import argparse
import shutil
import subprocess
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```python
from pathlib import Path

from yt_dlp import YoutubeDL

# When no folder is given, files go to a "downloads" folder next to this script.
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
# Video quality like YouTube's "720p": 360, 480, 720, 1080, etc. It is the
# short side of the picture (the height of a wide video).
DEFAULT_QUALITY = 720
# False: this quality or lower (or the nearest one above if nothing lower
# exists). True: exactly this quality, otherwise the video is skipped.
DEFAULT_EXACT_QUALITY = False
```

- `argparse` خط فرمان را می‌خواند، `shutil` به دنبال FFmpeg می‌گردد، `subprocess` یک بار FFmpeg را برای بررسی اجرا می‌کند و `Path` با پوشه‌ها کار می‌کند (فصل ۳).
- `YoutubeDL` کلاس دانلودر از `yt_dlp` است (فصل ۴).
- `SCRIPT_FOLDER` پوشه‌ای است که `downloader.py` در آن قرار دارد و `DOWNLOAD_FOLDER` پوشهٔ `downloads` در داخل آن است. عملگر `/` دو بخش یک مسیر را به هم وصل می‌کند. این پوشه با نخستین دانلود ساخته می‌شود.
- `DEFAULT_QUALITY` کیفیتی است که وقتی کیفیتی انتخاب نکنید به کار می‌رود: ۷۲۰، مانند 720p در یوتیوب. کیفیت یعنی **ضلع کوتاه** تصویر: ارتفاع ویدیوی افقی، اما عرض ویدیوی عمودی گوشی (فصل ۴).
- `DEFAULT_EXACT_QUALITY` تعیین می‌کند این عدد چقدر سخت‌گیرانه باشد. با `False` برنامه همین کیفیت یا نزدیک‌ترین کیفیت پایین‌تر را می‌گیرد.

نوشتن نام با حروف بزرگ، قراردادی در پایتون برای مقدارهایی است که هرگز تغییر نمی‌کنند (**ثابت‌ها**). نگه‌داشتن آن‌ها کنار هم در بالای فایل یعنی می‌توانید پیش‌فرض‌های برنامه را، مثلاً به ۱۰۸۰، در یک جای روشن تغییر دهید. دلیل ذخیرهٔ فایل‌ها در پوشهٔ جداگانهٔ `downloads` در فصل ۷ آمده است.

۳.۵ گام ۳: تشخیص پلی‌لیست

```python
def is_playlist(url):
    """Return True when the link contains a playlist id (list=...)."""
    return "list=" in url
```

هر پیوند پلی‌لیست یوتیوب عبارت `list=` و پس از آن شناسهٔ پلی‌لیست را دارد:

```
https://www.youtube.com/playlist?list=PLxxxxxxxx          -> playlist
https://www.youtube.com/watch?v=abc123&list=PLxxxxxxxx    -> playlist
https://www.youtube.com/watch?v=abc123                    -> one video
https://youtu.be/abc123                                   -> one video
```

عبارت `"list=" in url` خودش `True` یا `False` است؛ پس می‌توانیم آن را مستقیم برگردانیم. پیوند ویدیویی که **از داخل** یک پلی‌لیست باز شده باشد هم `list=` دارد و در این حالت کل پلی‌لیست دانلود می‌شود. برای دانلود فقط همان ویدیو، پیوند را بدون بخش `&list=...` کپی کنید.

۴.۵ گام ۴: انتخاب نام فایل‌ها

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```python
def file_name_template(url):
    """Tell yt-dlp how to name each downloaded file."""
    if is_playlist(url):
        # Playlist title/01 - Video title.mp4
        return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
    # Video title.mp4
    return "%(title)s.%(ext)s"
```

این تابع یکی از دو الگوی خروجی فصل ۴ را برمی‌گرداند. برای پلی‌لیست، نویسهٔ / پوشه‌ای به نام پلی‌لیست می‌سازد و شماره، ترتیب اصلی را حفظ می‌کند. برای یک ویدیو، فایل فقط به نام عنوانش ذخیره می‌شود. به return زودهنگام دقت کنید: اگر پیوند پلی‌لیست باشد، تابع در خط ۴۱ تمام می‌شود؛ پس به else نیازی نیست.

۵.۵ گام ۵: انتخاب کیفیت

```python
def video_quality(quality=DEFAULT_QUALITY, exact=DEFAULT_EXACT_QUALITY,
                  ffmpeg_location=None):
    """Build the yt-dlp format text for the requested quality.

    Quality means the short side of the picture, like YouTube's "720p": the
    height of a wide video, but the width of a vertical (phone) video.
    """
    if quality <= 0:
        raise ValueError("quality must be a positive number of pixels")
    has_ffmpeg = ffmpeg_location or shutil.which("ffmpeg")
    if not exact:
        # No filter here: "res:<quality>" in format_sort (see build_options)
        # picks the largest video up to this quality, or the smallest above it
        # when nothing smaller exists, so a video is never skipped for quality.
        return "bestvideo*+bestaudio/best" if has_ffmpeg else "best"
    # Exact: a wide video must be exactly this high, a vertical one this wide.
    wide = f"[height={quality}][width>={quality}]"
    tall = f"[width={quality}][height>={quality}]"
    if has_ffmpeg:
        # Best video + best audio, joined by ffmpeg; after each "/" comes the
        # next choice to try: a vertical video, then ready-made files.
        return (f"bestvideo*{wide}+bestaudio/bestvideo*{tall}+bestaudio/"
                f"best{wide}/best{tall}")
    # Without ffmpeg, only ready-made files that contain video and audio.
    return f"best{wide}/best{tall}"
```

این تابع متن format را برای yt-dlp برمی‌گرداند (فصل ۴).

۱. خط‌های ۵۳ و ۵۴ کیفیت صفر یا منفی را با ValueError رد می‌کنند، چون چنین عددی معنا ندارد. raise تابع را با آن خطا متوقف می‌کند (فصل ۳).

۲. خط ۵۵ به خاطر می‌سپارد که آیا FFmpeg در دسترس است: یا مسیری که کاربر داده، یا مسیری که shutil.which پیدا می‌کند. مقدار آن یک مسیر (درست) یا None (نادرست) است.

۳. خط‌های ۵۶ تا ۶۰، حالت عادی: قالب فقط «بهترین تصویر + بهترین صدا» (یا بدون FFmpeg فقط «بهترین») است. محدودیت کیفیت اینجا نوشته **نمی‌شود**؛ این محدودیت نخستین قانون format_sort در گام بعد است. آن قانون همیشه ویدیویی پیدا می‌کند، حتی وقتی کیفیت درخواستی وجود نداشته باشد.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۴. خط‌های ۶۱ تا ۷۰، حالت دقیق: دو فیلتر «دقیقاً همین کیفیت» را توصیف می‌کنند. `wide` برای ویدیوهای معمولی است که ارتفاعشان ضلع کوتاه است و `tall` برای ویدیوهای عمودی که عرضشان ضلع کوتاه است. قالب آن‌ها را یکی پس از دیگری امتحان می‌کند؛ `/` یعنی «اگر ممکن نبود، گزینهٔ بعدی را امتحان کن». اگر هیچ‌کدام جور نشود، `yt-dlp` پیام `Requested format is not available` را گزارش می‌کند و از این ویدیو می‌گذرد.

متن طولانی خط‌های ۶۷ و ۶۸ به شکل دو اف‌استرینگ داخل پرانتز نوشته شده است. پایتون رشته‌های کنار هم را خودکار به هم وصل می‌کند؛ این کار هر خط را کوتاه و خوانا نگه می‌دارد.

۶.۵ گام ۶: جمع‌کردن همهٔ تنظیم‌ها

```
۷۳ def build_options(url, folder, browser=None, audio=False, ffmpeg_location=None,
۷۴                   quality=DEFAULT_QUALITY, exact_quality=DEFAULT_EXACT_QUALITY):
۷۵     """Collect all settings in the dictionary that yt-dlp expects."""
۷۶     options = {
۷۷         "format": video_quality(quality, exact_quality, ffmpeg_location),
۷۸         # The ffmpeg program yt-dlp must use (None lets yt-dlp search by itself).
۷۹         "ffmpeg_location": ffmpeg_location or shutil.which("ffmpeg"),
۸۰         # Order of preference: the requested quality first ("res" is the short
۸۱         # side), then H264. video and AAC (m4a) audio: an mp4 that plays
۸۲         # everywhere.
۸۳         "format_sort": [f"res:{quality}", "vcodec:h264", "acodec:m4a"],
۸۴         "merge_output_format": "mp4",
۸۵         "paths": {"home": str(folder)},
۸۶         "outtmpl": file_name_template(url),
۸۷         # Skip a video that fails (private, deleted, failed merge) and continue
۸۸         # with the rest of the playlist instead of stopping.
۸۹         "ignoreerrors": True,
۹۰         # Retry after a network drop (the Python default is 0 retries).
۹۱         "retries": 10,
۹۲     }
۹۳     if audio:
۹۴         # Only the sound: the best audio stream; the video quality is ignored.
۹۵         options["format"] = "bestaudio/best"
۹۶         if options["ffmpeg_location"]:
۹۷             # ffmpeg converts the downloaded audio to a 192 kbit/s mp3 file.
۹۸             options["postprocessors"] = [
۹۹                 {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
۱۰۰                  "preferredquality": "192"},
۱۰۱             ]
۱۰۲     if browser:
۱۰۳         # Reuse the YouTube login of this browser when YouTube asks for it.
۱۰۴         options["cookiesfrombrowser"] = (browser,)
۱۰۵     return options
```

این تابع دیکشنری مورد انتظار `YoutubeDL` را می‌سازد. همهٔ کلیدها در جدول‌های فصل ۴ توضیح داده شده‌اند.

- `format` از تابعی می‌آید که همین حالا نوشتیم.
- `ffmpeg_location` به `yt-dlp` می‌گوید از کدام FFmpeg استفاده کند: همانی که کاربر داده، در غیر این صورت همانی که در PATH است. اگر None باشد، `yt-dlp` خودش جست‌وجو می‌کند.
- `format_sort` ترتیب ترجیح است. نخستین قانون آن، `res:720` (با عدد انتخاب‌شده)، بزرگ‌ترین

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

ویدیو تا آن کیفیت را ترجیح می‌دهد. سپس تصویر H.264 و صدای AAC ترجیح داده می‌شوند تا فایل `.mp4` نهایی همه‌جا پخش شود.

- `paths` پوشه را به شکل رشته (`str(folder)`) دریافت می‌کند، چون `yt-dlp` همین را انتظار دارد.
- `ignoreerrors` با مقدار `True` یعنی: وقتی یک ویدیو شکست بخورد (خصوصی، حذف‌شده، ترکیب ناموفق یا نبود کیفیت دقیق)، آن را گزارش کن، **از آن بگذر و ادامه بده** و بقیهٔ پلی‌لیست را دانلود کن.
- `retries` مهم است. **دستور** ترمینالی `yt-dlp` به‌طور پیش‌فرض ۱۰ بار دوباره تلاش می‌کند، اما مقدار پیش‌فرض آن در **پایتون** صفر است (فصل ۷).
- خط‌های ۹۳ تا ۱۰۱ فقط برای `--audio` اجرا می‌شوند. این خط‌ها قالب را با `bestaudio/best` (فقط صدا؛ تنظیم کیفیت اهمیتی ندارد) **جایگزین** می‌کنند و اگر FFmpeg باشد، یک **پس‌پردازنده** اضافه می‌کنند که نتیجه را به MP3 تبدیل می‌کند. بدون FFmpeg فایل صوتی اصلی (معمولاً `.m4a`) نگه داشته می‌شود.
- خط‌های ۱۰۲ تا ۱۰۴ ورود مرورگر را فقط وقتی اضافه می‌کنند که `--browser` داده شده باشد. `(browser,)` یک تاپل یک‌عضوی است.

ساختن دیکشنری در ابتدا و سپس تغییر چند کلید با بلوک‌های `if` یک الگوی رایج است: حالت عادی خوانا می‌ماند و هر حالت خاص یک بلوک کوچک و جداگانه است.

۷.۵ گام ۷: بررسی اینکه FFmpeg واقعاً کار می‌کند

```
108 def check_ffmpeg(location):
109     """Run "ffmpeg -version" once: finding the file does not prove it works."""
110     if not location:
111         return  # No ffmpeg: the program still works with ready-made files.
112     executable = Path(location).expanduser()
113     if executable.is_dir():
114         # A folder was given: look for the ffmpeg program inside it
115         # (shutil.which also finds "ffmpeg.exe" on Windows).
116         executable = executable / "ffmpeg"
117         executable = Path(shutil.which(str(executable)) or str(executable))
118     try:
119         result = subprocess.run(
120             [str(executable), "-version"], capture_output=True, text=True,
121             errors="replace", timeout=15,
122         )
123     except (OSError, subprocess.TimeoutExpired) as exc:
124         # OSError: the file is missing or not executable. Timeout: it hangs.
125         raise RuntimeError(f"Cannot run FFmpeg at {executable}: {exc}") from exc
126     if result.returncode:
127         # The program started but reported an error (for example a broken build
128         # or a missing system library). Show its own message to the user.
129         details = result.stderr.strip() or result.stdout.strip()
130         raise RuntimeError(
131             f"FFmpeg exists but cannot run: {executable} "
132             f"(exit code 0x{result.returncode & 0xffffffff:08X}). "
133             "Repair the FFmpeg installation or use --ffmpeg-location with a "
134             f"working build. {details}"
135         )
```

پیدا شدن فایلی به نام `ffmpeg` ثابت نمی‌کند که کار می‌کند: ممکن است نسخه‌ای خراب یا نیمه‌نصب باشد که هنگام شروع از کار می‌افتد. آن‌گاه هر دانلود درست در پایان کار، هنگام ترکیب تصویر و صدا، شکست

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

می‌خورد. این تابع پیش از هر دانلودی یک بار `ffmpeg -version` را اجرا می‌کند.

۱. خط‌های ۱۱۰ و ۱۱۱: بدون FFmpeg چیزی برای بررسی نیست. برنامه با فایل‌های آماده هم کار می‌کند؛ پس `return` فقط تابع را تمام می‌کند.

۲. خط‌های ۱۱۲ تا ۱۱۷: کاربر ممکن است به‌جای فایل برنامه، یک پوشه بدهد. آن‌گاه داخل آن به دنبال `ffmpeg` می‌گردیم. `shutil.which` با مسیر کامل، `ffmpeg.exe` را در ویندوز هم پیدا می‌کند.

۳. خط‌های ۱۱۸ تا ۱۲۲: `subprocess.run` برنامه را با آرگومان `-version` اجرا می‌کند. گزینهٔ `capture_output` متن خروجی را به‌جای چاپ نگه می‌دارد و `timeout` پس از ۱۵ ثانیه انتظار را متوقف می‌کند (فصل ۳).

۴. خط‌های ۱۲۳ تا ۱۲۵: خطای `OSError` یعنی فایل وجود ندارد یا اجرا نمی‌شود و `TimeoutExpired` یعنی برنامه گیر کرده است. هر دو به یک `RuntimeError` روشن تبدیل می‌شوند. `from exc` خطای اصلی را برای رفع اشکال پیوست نگه می‌دارد.

۵. خط‌های ۱۲۶ تا ۱۳۵: برنامه اجرا شده، اما یک **کد خروج** غیرصفر برگردانده که یعنی شکست. پیام شامل مسیر، کد خروج، توصیه و متن خطای خود FFmpeg است. کد خروج به شکل شانزده‌شانزدهی (`0x...`) چاپ می‌شود، چون ویندوز چنین خطاهایی را این‌گونه گزارش می‌کند؛ برای نمونه `0xC0000135` یعنی «یک DLL لازم پیدا نشد».

۸.۵ گام ۸: دانلود

```
138 def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False,
139              ffmpeg_location=None, quality=DEFAULT_QUALITY,
140              exact_quality=DEFAULT_EXACT_QUALITY):
141     """Download the link into folder. Return 0 when everything succeeded."""
142     # Use the ffmpeg the user gave, otherwise the one found on the PATH.
143     if ffmpeg_location:
144         location = str(Path(ffmpeg_location).expanduser())
145     else:
146         location = shutil.which("ffmpeg")
147     # Stop early with a clear message if that ffmpeg cannot run.
148     check_ffmpeg(location)
149 
150     folder = Path(folder).expanduser().resolve()
151     folder.mkdir(parents=True, exist_ok=True)
152     print(f"Saving to: {folder}")
153 
154     options = build_options(url, folder, browser, audio, location,
155                             quality, exact_quality)
156     with YoutubeDL(options) as ydl:
157         return ydl.download([url])
```

۱. خط‌های ۱۴۳ تا ۱۴۶: FFmpeg مورد استفاده را انتخاب می‌کنند: مسیری که کاربر با گزینهٔ `--ffmpeg-location` داده است، در غیر این صورت همانی که در PATH پیدا می‌شود. نویسهٔ ~ در این مسیر هم به پوشهٔ خانه تبدیل می‌شود.

۲. خط ۱۴۸ آن را بررسی می‌کند. اگر اجرا نشود، `check_ffmpeg` خطا ایجاد می‌کند و چیزی دانلود نمی‌شود.

۳. خط ۱۵۰ پوشهٔ ذخیره را به مسیر کامل تبدیل می‌کند: `expanduser` نویسهٔ ~ را با پوشهٔ خانه جایگزین می‌کند و `resolve` مسیر نسبی را مطلق می‌کند. خط ۱۵۱ آن را می‌سازد و خط ۱۵۲ آن را چاپ می‌کند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۴. خط‌های ۱۵۴ تا ۱۵۷ تنظیم‌ها را می‌سازند، دانلودر را ایجاد می‌کنند، پیوند را دانلود می‌کنند و نتیجهٔ `yt-dlp` را برمی‌گردانند: `0` وقتی همهٔ ویدیوها موفق باشند.

۹.۵ گام ۹: خواندن خط فرمان

```
۱۶۰ def main():
۱۶۱     parser = argparse.ArgumentParser(
۱۶۲         description="Download a YouTube video or a whole playlist."
۱۶۳     )
۱۶۴     parser.add_argument("url", nargs="?", help="video or playlist link")
۱۶۵     parser.add_argument(
۱۶۶         "-o", "--output", default=DOWNLOAD_FOLDER,
۱۶۷         help="folder for the files (default: downloads/ next to this script)",
۱۶۸     )
۱۶۹     parser.add_argument(
۱۷۰         "-a", "--audio", action="store_true",
۱۷۱         help="save only the sound, as an mp3 file",
۱۷۲     )
۱۷۳     parser.add_argument(
۱۷۴         "--browser",
۱۷۵         help="use this browser's YouTube login, e.g. firefox or chrome",
۱۷۶     )
۱۷۷     parser.add_argument(
۱۷۸         "--ffmpeg-location",
۱۷۹         help="path to a working ffmpeg executable or its containing folder",
۱۸۰     )
۱۸۱     parser.add_argument(
۱۸۲         "--quality", type=int, default=DEFAULT_QUALITY,
۱۸۳         help=f"video quality such as 480 or 720: this or lower "
۱۸۴              f"(default: {DEFAULT_QUALITY})",
۱۸۵     )
۱۸۶     parser.add_argument(
۱۸۷         "--exact-quality", action="store_true", default=DEFAULT_EXACT_QUALITY,
۱۸۸         help="require exactly this quality; skip the video if unavailable "
۱۸۹              "(ignored for audio)",
۱۹۰     )
۱۹۱     args = parser.parse_args()
۱۹۲     # type=int already rejects text; this also rejects 0 and negative numbers.
۱۹۳     if args.quality <= 0:
۱۹۴         parser.error("--quality must be a positive number, such as 720")
۱۹۵ 
۱۹۶     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
۱۹۷     url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
۱۹۸     if not url:
۱۹۹         parser.error("a link is required")
۲۰۰ 
۲۰۱     try:
۲۰۲         result = download(url, args.output, args.browser, args.audio,
۲۰۳                          args.ffmpeg_location, args.quality, args.exact_quality)
۲۰۴     except RuntimeError as exc:
۲۰۵         # A broken ffmpeg: print one clear line and exit with code 1.
۲۰۶         parser.exit(1, f"Error: {exc}\n")
۲۰۷     except KeyboardInterrupt:
۲۰۸         print("\nStopped. Run the same command again to continue.")
۲۰۹         return
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    if result == 0:
        print("Done. All files were downloaded.")
    else:
        print("Finished, but some files could not be downloaded (see the errors
    above).")
```

`main` کاربر را به تابع‌های بالا وصل می‌کند.

- `url` موقعیتی و اختیاری است (`nargs="?"`).
- `-o` پوشه را انتخاب می‌کند، `-a` حالت فقط صدا را روشن می‌کند (`action="store_true"`) و `--browser` نام مرورگری را برای کوکی‌ها می‌گیرد.
- `--ffmpeg-location` (خط ۱۷۸) خود برنامهٔ FFmpeg یا پوشهٔ آن را می‌پذیرد؛ برای وقتی که FFmpeg در PATH نیست یا نسخهٔ موجود در آن خراب است.
- `--quality` (خط ۱۸۲) از `type=int` استفاده می‌کند: `argparse` متن `"480"` را به عدد ۴۸۰ تبدیل می‌کند و متنی مانند `abc` را با پیامی روشن رد می‌کند.
- `--exact-quality` (خط ۱۸۷) یک کلید روشن/خاموش است.
- خط‌های ۱۹۳ و ۱۹۴ عددهای صفر و منفی را هم رد می‌کنند که `type=int` آن‌ها را می‌پذیرد.
- خط ۱۹۶ اگر پیوندی داده نشده باشد آن را می‌پرسد؛ خط ۱۹۷ بک‌اسلش‌هایی را که zsh هنگام جای‌گذاری اضافه می‌کند حذف می‌کند (فصل ۷)؛ خط‌های ۱۹۸ و ۱۹۹ با پیوند خالی متوقف می‌شوند.
- خط‌های ۲۰۱ تا ۲۰۹ دانلود را داخل `try` اجرا می‌کنند. خطای `RuntimeError` (یک FFmpeg خراب) در یک خط چاپ می‌شود و `parser.exit(1, ...)` برنامه را با کد خروج ۱ تمام می‌کند که به اسکریپت‌ها می‌گوید اجرا شکست خورده است. Ctrl+C (`KeyboardInterrupt`) یک جملهٔ دوستانه چاپ می‌کند.
- خط‌های ۲۱۱ تا ۲۱۴ بر اساس نتیجه، پیام پایانی را چاپ می‌کنند.

۱۰.۵ گام ۱۰: شروع برنامه

```
if __name__ == "__main__":
    main()
```

این دو خط فقط وقتی `main` را صدا می‌زنند که فایل مستقیم اجرا شده باشد، نه وقتی فایل آزمون آن را وارد می‌کند (فصل ۳).

۱۱.۵ برنامهٔ کامل

اکنون فایل شما باید دقیقاً مانند فهرست زیر باشد. اگر چیزی کار نکرد، آن‌ها را خط‌به‌خط مقایسه کنید.

Listing ۱.۵: The complete code in code/downloader.py

```
"""Download one YouTube video or a whole playlist with yt-dlp.

Examples:
    python downloader.py "VIDEO_OR_PLAYLIST_LINK"
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 480
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 1080 --exact-quality
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --ffmpeg-location /opt/ffmpeg/
    bin
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
    python downloader.py          # asks for the link
"""

import argparse
import shutil
import subprocess
from pathlib import Path

from yt_dlp import YoutubeDL

# When no folder is given, files go to a "downloads" folder next to this script.
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
# Video quality like YouTube's "720p": 360, 480, 720, 1080, etc. It is the
# short side of the picture (the height of a wide video).
DEFAULT_QUALITY = 720
# False: this quality or lower (or the nearest one above if nothing lower
# exists). True: exactly this quality, otherwise the video is skipped.
DEFAULT_EXACT_QUALITY = False


def is_playlist(url):
    """Return True when the link contains a playlist id (list=...)."""
    return "list=" in url


def file_name_template(url):
    """Tell yt-dlp how to name each downloaded file."""
    if is_playlist(url):
        # Playlist title/01 - Video title.mp4
        return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
    # Video title.mp4
    return "%(title)s.%(ext)s"


def video_quality(quality=DEFAULT_QUALITY, exact=DEFAULT_EXACT_QUALITY,
                  ffmpeg_location=None):
    """Build the yt-dlp format text for the requested quality.

    Quality means the short side of the picture, like YouTube's "720p": the
    height of a wide video, but the width of a vertical (phone) video.
    """
    if quality <= 0:
        raise ValueError("quality must be a positive number of pixels")
    has_ffmpeg = ffmpeg_location or shutil.which("ffmpeg")
    if not exact:
        # No filter here: "res:<quality>" in format_sort (see build_options)
        # picks the largest video up to this quality, or the smallest above it
        # when nothing smaller exists, so a video is never skipped for quality.
        return "bestvideo*+bestaudio/best" if has_ffmpeg else "best"
    # Exact: a wide video must be exactly this high, a vertical one this wide.
    wide = f"[height={quality}][width>={quality}]"
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    tall = f"[width={quality}][height>={quality}]"
    if has_ffmpeg:
        # Best video + best audio, joined by ffmpeg; after each "/" comes the
        # next choice to try: a vertical video, then ready-made files.
        return (f"bestvideo*{wide}+bestaudio/bestvideo*{tall}+bestaudio/"
                f"best{wide}/best{tall}")
    # Without ffmpeg, only ready-made files that contain video and audio.
    return f"best{wide}/best{tall}"


def build_options(url, folder, browser=None, audio=False, ffmpeg_location=None,
                  quality=DEFAULT_QUALITY, exact_quality=DEFAULT_EXACT_QUALITY):
    """Collect all settings in the dictionary that yt-dlp expects."""
    options = {
        "format": video_quality(quality, exact_quality, ffmpeg_location),
        # The ffmpeg program yt-dlp must use (None lets yt-dlp search by itself).
        "ffmpeg_location": ffmpeg_location or shutil.which("ffmpeg"),
        # Order of preference: the requested quality first ("res" is the short
        # side), then H264. video and AAC (m4a) audio: an mp4 that plays
        # everywhere.
        "format_sort": [f"res:{quality}", "vcodec:h264", "acodec:m4a"],
        "merge_output_format": "mp4",
        "paths": {"home": str(folder)},
        "outtmpl": file_name_template(url),
        # Skip a video that fails (private, deleted, failed merge) and continue
        # with the rest of the playlist instead of stopping.
        "ignoreerrors": True,
        # Retry after a network drop (the Python default is 0 retries).
        "retries": 10,
    }
    if audio:
        # Only the sound: the best audio stream; the video quality is ignored.
        options["format"] = "bestaudio/best"
        if options["ffmpeg_location"]:
            # ffmpeg converts the downloaded audio to a 192 kbit/s mp3 file.
            options["postprocessors"] = [
                {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
                 "preferredquality": "192"},
            ]
    if browser:
        # Reuse the YouTube login of this browser when YouTube asks for it.
        options["cookiesfrombrowser"] = (browser,)
    return options


def check_ffmpeg(location):
    """Run "ffmpeg -version" once: finding the file does not prove it works."""
    if not location:
        return  # No ffmpeg: the program still works with ready-made files.
    executable = Path(location).expanduser()
    if executable.is_dir():
        # A folder was given: look for the ffmpeg program inside it
        # (shutil.which also finds "ffmpeg.exe" on Windows).
        executable = executable / "ffmpeg"
        executable = Path(shutil.which(str(executable)) or str(executable))
    try:
        result = subprocess.run(
```


Alireza Khajehvandi صفحه‌های اجتماعی
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
            [str(executable), "-version"], capture_output=True, text=True,
            errors="replace", timeout=15,
        )
    except (OSError, subprocess.TimeoutExpired) as exc:
        # OSError: the file is missing or not executable. Timeout: it hangs.
        raise RuntimeError(f"Cannot run FFmpeg at {executable}: {exc}") from exc
    if result.returncode:
        # The program started but reported an error (for example a broken build
        # or a missing system library). Show its own message to the user.
        details = result.stderr.strip() or result.stdout.strip()
        raise RuntimeError(
            f"FFmpeg exists but cannot run: {executable} "
            f"(exit code 0x{result.returncode & 0xffffffff:08X}). "
            "Repair the FFmpeg installation or use --ffmpeg-location with a "
            f"working build. {details}"
        )


def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False,
             ffmpeg_location=None, quality=DEFAULT_QUALITY,
             exact_quality=DEFAULT_EXACT_QUALITY):
    """Download the link into folder. Return 0 when everything succeeded."""
    # Use the ffmpeg the user gave, otherwise the one found on the PATH.
    if ffmpeg_location:
        location = str(Path(ffmpeg_location).expanduser())
    else:
        location = shutil.which("ffmpeg")
    # Stop early with a clear message if that ffmpeg cannot run.
    check_ffmpeg(location)

    folder = Path(folder).expanduser().resolve()
    folder.mkdir(parents=True, exist_ok=True)
    print(f"Saving to: {folder}")

    options = build_options(url, folder, browser, audio, location,
                            quality, exact_quality)
    with YoutubeDL(options) as ydl:
        return ydl.download([url])


def main():
    parser = argparse.ArgumentParser(
        description="Download a YouTube video or a whole playlist."
    )
    parser.add_argument("url", nargs="?", help="video or playlist link")
    parser.add_argument(
        "-o", "--output", default=DOWNLOAD_FOLDER,
        help="folder for the files (default: downloads/ next to this script)",
    )
    parser.add_argument(
        "-a", "--audio", action="store_true",
        help="save only the sound, as an mp3 file",
    )
    parser.add_argument(
        "--browser",
        help="use this browser's YouTube login, e.g. firefox or chrome",
    )
```

```
    parser.add_argument(
        "--ffmpeg-location",
        help="path to a working ffmpeg executable or its containing folder",
    )
    parser.add_argument(
        "--quality", type=int, default=DEFAULT_QUALITY,
        help=f"video quality such as 480 or 720: this or lower "
             f"(default: {DEFAULT_QUALITY})",
    )
    parser.add_argument(
        "--exact-quality", action="store_true", default=DEFAULT_EXACT_QUALITY,
        help="require exactly this quality; skip the video if unavailable "
             "(ignored for audio)",
    )
    args = parser.parse_args()
    # type=int already rejects text; this also rejects 0 and negative numbers.
    if args.quality <= 0:
        parser.error("--quality must be a positive number, such as 720")

    url = args.url or input("Paste a YouTube video or playlist link: ").strip()
    url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
    if not url:
        parser.error("a link is required")

    try:
        result = download(url, args.output, args.browser, args.audio,
                          args.ffmpeg_location, args.quality, args.exact_quality)
    except RuntimeError as exc:
        # A broken ffmpeg: print one clear line and exit with code 1.
        parser.exit(1, f"Error: {exc}\n")
    except KeyboardInterrupt:
        print("\nStopped. Run the same command again to continue.")
        return

    if result == 0:
        print("Done. All files were downloaded.")
    else:
        print("Finished, but some files could not be downloaded (see the errors
    above).")


if __name__ == "__main__":
    main()
```

ایستگاه بررسی

در پوشهٔ code دستور `python downloader.py --help` را اجرا کنید. باید هفت آرگومان ببینید: `url`، `-o`، `-a`، `--browser`، `--ffmpeg-location`، `--quality` و `--exact-quality`. اگر پایتون خطای `SyntaxError` یا `IndentationError` نشان داد، خطی را که نام می‌برد با فهرست بالا مقایسه کنید.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۶

اجرا و آزمون برنامه

هدف فصل

یک ویدیو و یک پلی‌لیست را دانلود کنید، پوشهٔ ذخیره را انتخاب کنید و آزمون‌های آفلاین را اجرا کنید.

مطمئن شوید محیط فعال است (فصل ۲) و ترمینال در پوشهٔ `code` قرار دارد. در بعضی سیستم‌ها باید به‌جای `python` بنویسید `python3`.

۱.۶ نمایش صفحهٔ راهنما

```
python downloader.py --help
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

نتیجهٔ مورد انتظار

```
usage: downloader.py [-h] [-o OUTPUT] [-a] [--browser BROWSER]
                     [--ffmpeg-location FFMPEG_LOCATION] [--quality QUALITY]
                     [--exact-quality]
                     [url]

Download a YouTube video or a whole playlist.

positional arguments:
  url                   video or playlist link

options:
  -h, --help            show this help message and exit
  -o OUTPUT, --output OUTPUT
                        folder for the files (default: downloads/ next to
    this
                        script)
  -a, --audio           save only the sound, as an mp3 file
  --browser BROWSER     use this browser's YouTube login, e.g. firefox or
                        chrome
  --ffmpeg-location FFMPEG_LOCATION
                        path to a working ffmpeg executable or its
    containing
                        folder
  --quality QUALITY     video quality such as 480 or 720: this or lower
                        (default: 720)
  --exact-quality       require exactly this quality; skip the video if
                        unavailable (ignored for audio)
```

۲.۶ دانلود یک ویدیو

از نخستین ویدیویی که در یوتیوب بارگذاری شد استفاده می‌کنیم، چون فقط ۱۹ ثانیه است:

```
python downloader.py "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

`yt-dlp` کاری را که انجام می‌دهد و یک خط پیشرفت زنده چاپ می‌کند. خط‌های مهم به این شکل هستند (مسیرها کوتاه شده‌اند):

نتیجهٔ مورد انتظار

```
Saving to: /home/sara/prj-youtube-downloader/code/downloads
[youtube] Extracting URL: https://www.youtube.com/watch?v=jNQXAC9IVRw
[info] jNQXAC9IVRw: Downloading 1 format(s): 18
[download] Destination: .../code/downloads/Me at the zoo.mp4
[download] 100% of  ...
Done. All files were downloaded.
```

اکنون فایل `Me at the zoo.mp4` در پوشهٔ `code/downloads/` قرار دارد. این ویدیوی قدیمی نسخهٔ HD ندارد؛ پس یک فایل آماده (قالب 18) کافی بود. برای یک ویدیوی HD دو شمارهٔ قالب، مانند 140+137، و یک خط اضافهٔ `[Merger] Merging formats into ...` می‌بینید؛ این همان FFmpeg است که تصویر و صدای جدا را ترکیب می‌کند. شمارهٔ قالب‌ها ممکن است روی رایانهٔ شما فرق کند؛ این طبیعی است.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۳.۶ دانلود یک پلی‌لیست کامل

یک پلی‌لیست یوتیوب را باز کنید، پیوند را از نوار نشانی کپی کنید و اجرا کنید:

```
python downloader.py "https://www.youtube.com/playlist?list=PLAYLIST_ID"
```

نتیجهٔ مورد انتظار

پوشه‌ای تازه به نام پلی‌لیست در code/downloads/ ساخته می‌شود. ویدیوهای داخل آن به ترتیب پلی‌لیست شماره‌گذاری شده‌اند و عنوان دقیق خود را نگه می‌دارند؛ برای نمونه:

```
Get more creative with AI/
|-- 01 - Vibe coding in Google AI Studio is changing the game.mp4
|-- 02 - 3 Incredible #NanoBanana Community Builds.mp4
`-- ...
```

دانلود یک پلی‌لیست بزرگ ممکن است زمان زیادی بگیرد. هر لحظه می‌توانید با Ctrl+C آن را متوقف کنید. اگر بعداً همان دستور را دوباره اجرا کنید، yt-dlp ویدیوهای کامل‌شده را رد می‌کند و بقیه را ادامه می‌دهد.

۴.۶ ذخیرهٔ فقط صدا

گزینهٔ a- یا audio-- را اضافه کنید. این گزینه هم برای یک ویدیو و هم برای پلی‌لیست کار می‌کند:

```
python downloader.py "https://youtu.be/huwi-cjPPXU" --audio
```

نتیجهٔ مورد انتظار

```
[info] huwi-cjPPXU: Downloading 1 format(s): 140
[download] 100% of   00.14MiB in 00:00:04 at 98.2MiB/s
[ExtractAudio] Destination: .../code/downloads/Introduce Yourself
               in Slow German (for Absolute Beginners).mp3
Deleting original file ... .m4a (pass -k to keep)
Done. All files were downloaded.
```

قالب 140 صدای AAC یوتیوب است. خط ExtractAudio همان FFmpeg است که آن را به MP3 تبدیل می‌کند؛ سپس فایل موقت m4a. پاک می‌شود. این درس ۱۵ دقیقه‌ای به یک فایل MP3 ۲۱ مگابایتی تبدیل می‌شود و به‌جای ۱۴۴ مگابایتِ ویدیوی Full HD فقط ۱۴ مگابایت دانلود لازم دارد.

۵.۶ انتخاب کیفیت ویدیو

بدون quality-- برنامه کیفیت 720p یا نزدیک‌ترین کیفیت پایین‌تر را ذخیره می‌کند. نتیجه‌های واقعی زیر از یک ویدیوی ۵۳ ثانیه‌ای **عمودی** به دست آمده‌اند (ویدیوی گوشی؛ پس ضلع کوتاه آن عرض است):

```
python downloader.py "LINK"                                    # default: 720
python downloader.py "LINK" --quality 480
python downloader.py "LINK" --quality 1080 --exact-quality
python downloader.py "LINK" --quality 2160 --exact-quality
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

نتیجهٔ مورد انتظار

```
[info] KXaSULPslqM: Downloading 1 format(s): 136+140-20      -> 720x1280
[info] KXaSULPslqM: Downloading 1 format(s): 135+140-20      -> 480x854
[info] KXaSULPslqM: Downloading 1 format(s): 137+140-20      -> 1080x1920
ERROR: [youtube] KXaSULPslqM: Requested format is not available. ...
Finished, but some files could not be downloaded (see the errors above).
```

ابعاد سمت راست پس از دانلود با `ffprobe` اندازه‌گیری شده‌اند. این ویدیو نسخهٔ 2160p (4K) ندارد؛ پس با `--exact-quality` به‌جای توقف برنامه، با پیامی روشن از آن گذشت. در یک پلی‌لیست، ویدیوهای دیگر همچنان دانلود می‌شدند. بدون `--exact-quality` نبود یک کیفیت هرگز مشکلی ایجاد نمی‌کند: برنامه نزدیک‌ترین کیفیت پایین‌تر را می‌گیرد، یا اگر پایین‌تری نبود، کوچک‌ترین کیفیت بالاتر را. گزینهٔ `--quality` همراه با `--audio` اثری ندارد.

۶.۶ استفاده از یک FFmpeg مشخص

برنامه معمولاً FFmpeg را خودش پیدا می‌کند. اگر FFmpeg جای دیگری نصب شده باشد، خود فایل برنامه یا پوشهٔ آن را با `--ffmpeg-location` بدهید:

```
python downloader.py "LINK" --ffmpeg-location /usr/bin                # Linux
    folder
python downloader.py "LINK" --ffmpeg-location "C:\ffmpeg\bin"         # Windows
    folder
python downloader.py "LINK" --ffmpeg-location ~/tools/ffmpeg          # program
    file
```

برنامه پیش از دانلود هر چیزی، یک بار `ffmpeg -version` را اجرا می‌کند. مسیر اشتباه یا FFmpeg خراب، برنامه را بی‌درنگ با یک خط روشن و کد خروج ۱ متوقف می‌کند، به‌جای اینکه بعداً روی تک‌تک ویدیوها شکست بخورد:

نتیجهٔ مورد انتظار

```
Error: Cannot run FFmpeg at /no/such/ffmpeg: [Errno 2] No such file or
directory: '/no/such/ffmpeg'

Error: FFmpeg exists but cannot run: .../ffmpeg (exit code 0x0000007F).
Repair the FFmpeg installation or use --ffmpeg-location with a working
build. libfoo.so: cannot open shared object file
```

پیام دوم از یک برنامهٔ آزمایشیِ عمداً خراب آمده که نمی‌تواند یک کتابخانه را بارگذاری کند. انتهای پیام، توضیح خود FFmpeg است.

۷.۶ انتخاب پوشهٔ ذخیره

گزینهٔ `-o` و یک پوشه را اضافه کنید. اگر پوشه وجود نداشته باشد، ساخته می‌شود:

```
python downloader.py "LINK" -o ~/Videos              # macOS / Linux
python downloader.py "LINK" -o "D:\My Videos"        # Windows
python downloader.py "LINK" -o downloads             # a folder here
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

خط اول خروجی، یعنی `Saving to: ...`، همیشه مسیر کامل را نشان می‌دهد. اگر نام پوشه فاصله دارد، آن را داخل گیومه بنویسید.

۸.۶ روش‌های دیگر اجرا

```
python downloader.py                          # asks for the link
python downloader.py "LINK" --browser firefox  # use Firefox's YouTube login
python downloader.py "LINK" --browser chrome -o ~/Videos
```

بدون پیوند، برنامه پیام `Paste a YouTube video or playlist link:` را نشان می‌دهد. پیوند را جای‌گذاری کنید (در ترمینال لینوکس معمولاً با Ctrl+Shift+V و در ویندوز با کلیک راست) و Enter را بزنید. گزینهٔ `--browser` را فقط وقتی به کار ببرید که یوتیوب از شما بخواهد وارد شوید (فصل ۸). نام‌های پشتیبانی‌شده شامل `firefox`، `chrome`، `chromium`، `edge`، `brave`، `opera` و `safari` هستند.

۹.۶ توقف و ادامه در زمانی دیگر

هر لحظه می‌توانید Ctrl+C را بزنید. برنامه پاسخ می‌دهد:

نتیجهٔ مورد انتظار

```
Stopped. Run the same command again to continue.
```

بخش نیمه‌کاره به شکل یک فایل `.part` در پوشه می‌ماند. وقتی همان دستور را دوباره اجرا کنید، `yt-dlp` پیام `Resuming download at byte ...` را چاپ می‌کند و به‌جای شروع دوباره، از همان‌جا ادامه می‌دهد.

۱۰.۶ آزمون‌های آفلاین

آزمون‌ها برنامه‌های کوچکی هستند که کدهای دیگر را خودکار بررسی می‌کنند. آزمون‌های ما در کسری از ثانیه اجرا می‌شوند و به اینترنت نیاز ندارند، چون فقط بررسی می‌کنند برنامه دانلود را چگونه **آماده** می‌کند. فایل `test_downloader.py` را کنار `downloader.py` بسازید:

Listing ۱.۶: The complete code in code/test_downloader.py

```
"""Offline checks for downloader.py. Run with: python -m unittest -v"""

import subprocess
import unittest
from pathlib import Path
from unittest.mock import patch

import downloader

VIDEO = "https://www.youtube.com/watch?v=abc123"
PLAYLIST = "https://www.youtube.com/playlist?list=PL123"


class DownloaderTests(unittest.TestCase):
    def test_detects_playlist_and_single_video(self):
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
        self.assertTrue(downloader.is_playlist(PLAYLIST))
        self.assertFalse(downloader.is_playlist(VIDEO))

    def test_single_video_is_named_by_title(self):
        self.assertEqual(downloader.file_name_template(VIDEO), "%(title)s.%(ext)s"
    )

    def test_playlist_gets_its_own_folder(self):
        name = downloader.file_name_template(PLAYLIST)
        self.assertTrue(name.startswith("%(playlist_title)s/"))

    def test_default_folder_is_downloads_next_to_script(self):
        script_folder = Path(downloader.__file__).resolve().parent
        self.assertEqual(downloader.DOWNLOAD_FOLDER, script_folder / "downloads")

    def test_options_use_given_folder_and_browser(self):
        options = downloader.build_options(VIDEO, "/tmp/videos", "firefox")
        self.assertEqual(options["paths"], {"home": "/tmp/videos"})
        self.assertEqual(options["cookiesfrombrowser"], ("firefox",))

    def test_retries_after_network_drop(self):
        self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"
    ], 10)

    def test_audio_only_becomes_mp3(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
        self.assertEqual(options["format"], "bestaudio/best")
        self.assertEqual(options["postprocessors"][0]["preferredcodec"], "mp3")

    def test_audio_only_without_ffmpeg_keeps_original_file(self):
        with patch("shutil.which", return_value=None):
            options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
        self.assertEqual(options["format"], "bestaudio/best")
        self.assertNotIn("postprocessors", options)

    def test_quality_without_ffmpeg(self):
        with patch("shutil.which", return_value=None):
            self.assertEqual(downloader.video_quality(), "best")

    def test_default_quality_prefers_720_or_lower(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            options = downloader.build_options(VIDEO, "/tmp/videos")
        self.assertEqual(options["format"], "bestvideo*+bestaudio/best")
        self.assertEqual(options["format_sort"][0], "res:720")

    def test_quality_option_changes_the_preference(self):
        options = downloader.build_options(VIDEO, "/tmp/videos", quality=480)
        self.assertEqual(options["format_sort"][0], "res:480")

    def test_exact_quality_works_for_wide_and_vertical_videos(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            text = downloader.video_quality(1080, exact=True)
        self.assertIn("bestvideo*[height=1080][width>=1080]+bestaudio", text)  #
    wide
        self.assertIn("bestvideo*[width=1080][height>=1080]+bestaudio", text)  #
    vertical
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    def test_exact_quality_without_ffmpeg_uses_ready_made_files(self):
        with patch("shutil.which", return_value=None):
            text = downloader.video_quality(360, exact=True)
        self.assertEqual(text, "best[height=360][width>=360]/best[width=360][height>=360]")

    def test_quality_must_be_positive(self):
        with self.assertRaises(ValueError):
            downloader.video_quality(0)

    def test_given_ffmpeg_location_is_used(self):
        with patch("shutil.which", return_value=None):
            options = downloader.build_options(VIDEO, "/tmp/videos",
                                               ffmpeg_location="/opt/ffmpeg")
        self.assertEqual(options["ffmpeg_location"], "/opt/ffmpeg")
        self.assertTrue(options["format"].startswith("bestvideo*"))

    def test_no_ffmpeg_needs_no_check(self):
        self.assertIsNone(downloader.check_ffmpeg(None))

    def test_missing_ffmpeg_file_is_reported(self):
        with self.assertRaises(RuntimeError):
            downloader.check_ffmpeg("/no/such/folder/ffmpeg")

    def test_broken_ffmpeg_is_reported(self):
        broken = subprocess.CompletedProcess([], returncode=1, stdout="", stderr="bad build")
        with patch("subprocess.run", return_value=broken):
            with self.assertRaises(RuntimeError):
                downloader.check_ffmpeg("/usr/bin/ffmpeg")


if __name__ == "__main__":
    unittest.main()
```

- `unittest` ماژول آزمون داخلی پایتون است. هر متدی داخل یک کلاس `unittest.TestCase` که نامش با `test_` شروع شود، یک آزمون است.
- `assertTrue`، `assertFalse` و `assertEqual` یک نتیجه را بررسی می‌کنند؛ اگر بررسی نادرست باشد، آزمون شکست می‌خورد و هر دو مقدار را نشان می‌دهد.
- `patch` یک تابع را موقتاً جایگزین می‌کند. در آزمون آخر، `shutil.which` وانمود می‌کند FFmpeg وجود ندارد؛ پس می‌توانیم آن حالت را حتی روی رایانه‌ای که FFmpeg دارد هم آزمون کنیم. به همین شکل، `test_broken_ffmpeg_is_reported` تابع `subprocess.run` را با یک نتیجهٔ ساختگی با کد خروج ۱ جایگزین می‌کند؛ پس FFmpeg خراب را بدون داشتن آن آزمون می‌کنیم. دو آزمون صدا همین ترفند را در هر دو جهت به کار می‌برند: وجود FFmpeg (تبدیل به MP3 اضافه می‌شود) و نبود آن (فایل صوتی اصلی نگه داشته می‌شود).

همهٔ آزمون‌ها را اجرا کنید:

```
python -m unittest -v
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

نتیجهٔ مورد انتظار

```
test_audio_only_becomes_mp3 ... ok
test_audio_only_without_ffmpeg_keeps_original_file ... ok
test_broken_ffmpeg_is_reported ... ok
test_default_folder_is_downloads_next_to_script ... ok
test_default_quality_prefers_720_or_lower ... ok
test_detects_playlist_and_single_video ... ok
test_exact_quality_without_ffmpeg_uses_ready_made_files ... ok
test_exact_quality_works_for_wide_and_vertical_videos ... ok
test_given_ffmpeg_location_is_used ... ok
test_missing_ffmpeg_file_is_reported ... ok
test_no_ffmpeg_needs_no_check ... ok
test_options_use_given_folder_and_browser ... ok
test_playlist_gets_its_own_folder ... ok
test_quality_must_be_positive ... ok
test_quality_option_changes_the_preference ... ok
test_quality_without_ffmpeg ... ok
test_retries_after_network_drop ... ok
test_single_video_is_named_by_title ... ok
----------------------------------------------------------------------
Ran 18 tests in 001.0s

OK
```

هر بار که `downloader.py` را تغییر دادید، آزمون‌ها را دوباره اجرا کنید. اگر رفتاری را عمداً تغییر دادید، آزمون متناظر را هم به‌روز کنید.

ایستگاه بررسی

یک ویدیو را در پوشهٔ `code/downloads` دانلود کرده‌اید، یک فایل صوتی را با `--audio` ذخیره کرده‌اید، یک پلی‌لیست (یا بخشی از آن) را در پوشهٔ خودش دانلود کرده‌اید، از هر کدام از `-o` و `--quality` یک بار استفاده کرده‌اید و پیام `OK` را از آزمون‌ها دیده‌اید.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۷

درس‌هایی از استفادهٔ واقعی

هدف فصل

ببینید نسخهٔ نخست برنامه پس از استفاده روی ویدیوهای واقعی چگونه بهتر شد: چه چیزی خراب شد، علت آن چگونه پیدا شد، چرا هر اصلاح درست کار می‌کند و چگونه آزمون شد. این‌ها عادت‌هایی هستند که یک اسکریپتِ کارکن را به ابزاری قابل‌اعتماد تبدیل می‌کنند.

نسخهٔ نخست `downloader.py` همهٔ آزمون‌هایش را با موفقیت گذراند و روی ویدیوهای کوتاه کار کرد. سپس استفادهٔ واقعی مشکل‌هایی را نشان داد که هیچ آزمونی آن‌ها را پوشش نمی‌داد. این فصل همان داستان را تعریف می‌کند، چون **پیدا کردن** یک مشکل از هر کد آماده‌ای آموزنده‌تر است. همهٔ تغییرهایی که اینجا توضیح داده می‌شوند، از قبل در برنامهٔ فصل ۵ وجود دارند.

۱.۷ روشی ساده برای هر باگ

۱. **نشانه را دقیق توصیف کنید.** چه انتظاری داشتید و چه اتفاقی افتاد؟

۲. **کل خروجی را بخوانید**، به‌ویژه نخستین هشدار یا خطا را.

۳. **شواهد را بررسی کنید**: کدام فایل‌ها وجود دارند، چه حجمی دارند و نامشان چه می‌گوید.

۴. **دربارهٔ علت حدس بزنید و سپس آن را بررسی کنید**؛ در مستندات یا کد منبع کتابخانه. فقط بر اساس حدس، کد را تغییر ندهید.

۵. **علت را اصلاح کنید، نه نشانه را**؛ با کوچک‌ترین تغییر روشن.

۶. **یک آزمون اضافه کنید** که این مشکل را پیدا می‌کرد و حالت واقعی را دوباره امتحان کنید.

۲.۷ درس ۱: قطع شبکه نیمی از ویدیو را جا گذاشت

۱.۲.۷ نشانه

یک کاربر یک درس ۱۵ دقیقه‌ای با کیفیت Full HD را با این دستور دانلود کرد:

```
python downloader.py "https://youtu.be/huwi-cjPPXU\?si\=Huhcwi9PkWR87c-u" -o ~/
    Videos
```


صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

خروجی یک خطا داشت و برنامه با پیام «برخی فایل‌ها دانلود نشدند» تمام شد:

```
[info] huwi-cjPPXU: Downloading 1 format(s): 137+140
[download] Destination: ...Beginners).f137.mp4
[download] Got error: 9849475 bytes read, 318544 more expected:
[download] Destination: ...Beginners).f140.m4a
[download] 100% of   00.14MiB in 00:01:58 at 43.121KiB/s
Finished, but some files could not be downloaded (see the errors above).
```

در پوشه یک فایل نیمه‌کارهٔ `.f137.mp4.part` بود که تصویر داشت اما **صدا نداشت** و یک فایل `.f140.m4a` که **فقط صدا** داشت. به نظر می‌رسید برنامه ویدیو را دو بار و در دو قالب اشتباه دانلود کرده است.

۲.۲.۷ خواندن شواهد

- `140+137` یعنی دو قالب: 137 جریان تصویر 1080p و 140 جریان صدای AAC است (فصل ۴). دانلود دو فایل **طبیعی** است؛ FFmpeg آن‌ها را در پایان کار ترکیب می‌کند. پس پرسش واقعی این است: چرا ترکیبی انجام نشد؟
- `Got error: ... more expected` یعنی اتصال پیش از کامل شدن بخش تصویر بسته شده است. سرعت (121 KiB/s) هم نشان می‌دهد شبکه کند و ناپایدار بوده است.
- هیچ خط `Retrying (1/10)` دیده نمی‌شود. دانلود در نخستین خطا رها شده و بدون بخش کامل تصویر، FFmpeg چیزی برای ترکیب نداشته است.

۳.۲.۷ پیدا کردن علت

همه می‌دانند **دستور** ترمینالی `yt-dlp` ۱۰ بار دوباره تلاش می‌کند. پس چرا اینجا نه؟ پاسخ در کد منبع کتابخانه است که pip آن را در پوشهٔ `site-packages/yt_dlp` محیط نصب کرده است:

```
# options.py -- only used by the yt-dlp terminal command
'--retries', dest='retries', default=10, ...

# downloader/http.py -- used by everyone, including our program
for retry in RetryManager(self.params.get('retries'), self.report_retry):
```

مقدار پیش‌فرض ۱۰ متعلق به **تجزیه‌گر خط فرمان** است. وقتی YoutubeDL را از پایتون به کار می‌بریم، `params.get('retries')` چیزی پیدا نمی‌کند و None برمی‌گرداند که یعنی **صفر** بار تلاش دوباره.

نکتهٔ مهم

کتابخانه‌ای که از پایتون به کار می‌رود، همیشه مانند همان ابزار در ترمینال رفتار نمی‌کند. مقدارهای پیش‌فرض اغلب در لایهٔ خط فرمان تعریف می‌شوند. وقتی تنظیمی مهم است، به‌جای تکیه بر مقدار پیش‌فرض، آن را صریح در تنظیم‌های خود بنویسید.

۴.۲.۷ اصلاح و آزمون

یک خط در `build_options` (فصل ۵، گام ۶):

```
"retries": 10,
```

یک آزمون تازه از آن محافظت می‌کند تا کسی آن را اشتباهی حذف نکند:

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
def test_retries_after_network_drop(self):
    self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"],
    10)
```

سپس حالت واقعی دوباره امتحان شد. همان دستور فایل نیمه‌کاره را ادامه داد:

```
[download] Resuming download at byte 49650475
```

سپس صدا را دانلود کرد و FFmpeg آن‌ها را ترکیب کرد. بررسی با `ffprobe` (که همراه FFmpeg نصب می‌شود) نتیجه را ثابت کرد:

```
stream|codec_name=h264|codec_type=video|width=1920|height=1080|duration64.906=
stream|codec_name=aac|codec_type=audio|duration69.906=
```

هر دو جریان وجود دارند و هر دو کامل، یعنی ۱۵ دقیقه و ۶ ثانیه، هستند.

۳.۷ درس ۲: ترمینال پیوند را تغییر داد

۱.۳.۷ نشانه

دوباره به پیوند دستور بالا نگاه کنید: `=\si?\.` کاربر یک پیوند معمولی را جای‌گذاری کرده بود. این بک‌اسلش‌ها را zsh، پوستهٔ پیش‌فرض مک و بسیاری از لینوکس‌ها، اضافه کرده بود. هنگام جای‌گذاری یک نشانی، قابلیتی از zsh به نام url-quote-magic از نویسه‌های ویژه‌ای مانند ? و = با یک بک‌اسلش محافظت می‌کند؛ کاری که **بدون** گیومه درست است. اما داخل گیومه، بک‌اسلش‌ها بخشی از متن می‌شوند و پایتون یک پیوند خراب دریافت می‌کند.

۲.۳.۷ چرا مهم است

این ویدیو از روی شانس باز هم کار کرد، چون `yt-dlp` شناسهٔ ویدیو را پیش از بخش خراب پیدا کرد. اما برای پیوند پلی‌لیست بی‌صدا شکست می‌خورد: `list=\` شامل `list=` نیست؛ پس `is_playlist` مقدار `False` برمی‌گرداند و کل پلی‌لیست بدون پوشه و شماره ذخیره می‌شد.

۳.۳.۷ اصلاح

نشانی معتبر هرگز بک‌اسلش ندارد؛ پس همیشه حذف آن‌ها درست در لحظهٔ خواندن پیوند بی‌خطر است (فصل ۵، گام ۸):

```
url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
```

درس این بخش: **ورودی کاربر را همان جایی که وارد برنامه می‌شود تمیز کنید.** آن‌گاه همهٔ تابع‌های بعدی می‌توانند به آن مقدار اعتماد کنند.

۴.۷ درس ۳: دانلودها را از کد و از گیت جدا نگه دارید

در نسخهٔ نخست، فایل‌ها بدون `-o` درست کنار `downloader.py` ذخیره می‌شدند. پس از چند دانلود، پوشهٔ `code` پر از ویدیوهای حجیم شده بود که با فایل‌های کد قاطی بودند و یک `git add .` بی‌دقت می‌توانست بخواهد گیگابایت‌ها داده را در گیت‌هاب بارگذاری کند.

اکنون پیش‌فرض یک پوشهٔ جداگانه است:

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
```

و فایل `.gitignore` خط `code/downloads/` را دارد. می‌توانید از گیت بپرسید آیا فایلی نادیده گرفته می‌شود و با کدام قانون:

```
git check-ignore -v "code/downloads/Me at the zoo.mp4"
```

```
.gitignore:222:code/downloads/  code/downloads/Me at the zoo.mp4
```

اکنون دو محافظت مستقل وجود دارد: قانون پوشه و قانون‌های قدیمی‌تر برای پسوندهای رسانه‌ای مانند `*.mp4`. جدا کردن فایل‌های **برنامه** از فایل‌های **داده** عادتی خوب در هر پروژه‌ای است.

۵.۷ درس ۴: یک قابلیت تازه، فقط صدا

زبان‌آموزان، شنوندگان پادکست و دوستداران موسیقی اغلب فقط به صدا نیاز دارند. دانلود ویدیوی Full HD برای این کار وقت و فضا را هدر می‌دهد: درس بالا به شکل ویدیو ۱۴۴ مگابایت است، اما صدایش فقط ۱۴ مگابایت دانلود لازم دارد.

طراحی، برنامه را کوچک نگه می‌دارد:

- یک کلید در خط فرمان: `-a` یا `--audio` با `action="store_true"` (فصل ۳).
- یک پارامتر تازه، `audio=False`، که از `main` از طریق `download` به `build_options` می‌رسد. مقدار پیش‌فرض آن رفتار قبلی را حفظ می‌کند؛ پس دستورهای قبلی هیچ تغییری نمی‌کنند.
- یک بلوک `if audio:` که فقط چیزهای متفاوت را تغییر می‌دهد. قالب به `bestaudio/best` تبدیل می‌شود و یک پس‌پردازندهٔ FFmpeg نتیجه را به MP3 تبدیل می‌کند (فصل ۴).
- اگر FFmpeg نباشد، پس‌پردازنده اضافه **نمی‌شود**، چون شکست می‌خورد. به‌جای آن فایل صوتی اصلی نگه داشته می‌شود. این همان ایدهٔ «با امکانات همین رایانه کار کن» در `video_quality` است.

دو آزمون هر دو حالت را پوشش می‌دهند. آن‌ها با `patch` وانمود می‌کنند FFmpeg وجود دارد یا ندارد؛ پس روی هر رایانه‌ای نتیجهٔ یکسانی می‌دهند.

۶.۷ درس ۵: توقف مؤدبانه

زدن Ctrl+C در میانهٔ دانلود، پیش‌تر یک گزارش خطای طولانی چاپ می‌کرد که شبیه خرابی برنامه بود. اکنون `main` استثنای `KeyboardInterrupt` را می‌گیرد (فصل ۳) و یک جملهٔ روشن چاپ می‌کند:

```
Stopped. Run the same command again to continue.
```

این جمله به دلیل روش کار `yt-dlp` درست است: دانلود ناتمام به شکل فایل `.part` نگه داشته می‌شود و اجرای بعدی آن را ادامه می‌دهد. پیام خوب به کاربر می‌گوید **چه اتفاقی افتاد** و **گام بعدی چیست**.

۷.۷ درس ۶: انتخاب کیفیت و ویدیوی عمودی

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

۱.۷.۷ نیاز و نسخهٔ نخست

فایل‌های Full HD حجیم هستند و روی اتصال ضعیف کند دانلود می‌شوند و اغلب کیفیت 720p کافی است. پس برنامه گزینهٔ `--quality` را گرفت. نسخهٔ نخست روی **ارتفاع** تصویر فیلتر می‌گذاشت:

```
bestvideo*[height<=720]+bestaudio/best[height<=720]
```

آزمون‌هایش موفق بودند، چون فقط بررسی می‌کردند که این متن درست ساخته شود.

۲.۷.۷ نشانه

نخستین دانلود واقعی روی یک ویدیوی کوتاهِ **عمودی** انجام شد. با پیش‌فرض ۷۲۰ نتیجه فقط ۶۴۰×۳۶۰ بود و `--quality 480` ابعاد ۴۲۶×۲۴۰ داد. فهرست قالب‌ها با `yt-dlp -F` دلیل را نشان داد:

```
18      mp4   360x640     30  2 |  ... avc142001.E
136     mp4   720x1280    30    |  ... avc14.d401f
137     mp4   1080x1920   30    |  ... avc1640028.
```

نسخهٔ 720p (قالب ۱۳۶) وجود دارد، اما **ارتفاع** آن ۱۲۸۰ است؛ پس `[height<=720]` آن را رد کرد. یوتیوب کیفیت را بر اساس **ضلع کوتاه** تصویر نام‌گذاری می‌کند و در ویدیوی عمودی، ضلع کوتاه عرض است. حالت دقیق هم همین باگ را داشت: `[height=1080]` یک ویدیوی ۱۰۸۰×۶۰۸ را انتخاب کرد که در واقع «608p» است.

۳.۷.۷ اصلاح

- حالت عادی دیگر فیلتر ندارد. محدودیت به نخستین قانون `format_sort` تبدیل شد: `res:720`. در `yt-dlp`، `res` **همان** ضلع کوتاه است و یک قانون مرتب‌سازی هرگز ویدیویی را رد نمی‌کند: بزرگ‌ترین ویدیو تا ۷۲۰ را می‌گیرد، یا اگر کوچک‌تری نبود، کوچک‌ترین ویدیوی بالاتر را.
- حالت دقیق به فیلتر نیاز دارد؛ پس از دو فیلتر استفاده می‌کند: `[height=Q][width>=Q]` برای ویدیوهای افقی و `[width=Q][height>=Q]` برای ویدیوهای عمودی که یکی پس از دیگری امتحان می‌شوند.

پس از اصلاح، همان ویدیو ابعاد ۱۲۸۰×۷۲۰، ۸۵۴×۴۸۰ و ۱۹۲۰×۱۰۸۰ داد و یک ویدیوی افقی ۷۲۰×۱۲۸۰ داد. آزمون‌های تازه هر دو جهت را بررسی می‌کنند.

نکتهٔ مهم

آزمونی که فقط بررسی می‌کند یک متن «طبق برنامه» ساخته شده، نمی‌تواند برنامهٔ اشتباه را پیدا کند. هر قابلیت تازه را همیشه روی ورودی‌های واقعی و گوناگون امتحان کنید: افقی و عمودی، قدیمی و جدید، کوتاه و بلند.

۸.۷ درس ۷: پیدا شدن FFmpeg با کارکردن آن یکی نیست

`shutil.which("ffmpeg")` فقط ثابت می‌کند فایلی با این نام وجود دارد. به‌ویژه در ویندوز، یک FFmpeg کپی‌شده یا نیمه‌نصب می‌تواند وجود داشته باشد و باز هم هنگام شروع از کار بیفتد؛ برای نمونه با کد خروج `0xC0000135` («یک DLL لازم پیدا نشد»). بدون بررسی، تک‌تک ویدیوها کامل دانلود می‌شدند و سپس در آخرین گام، یعنی ترکیب، شکست می‌خوردند و زمان و حجم اینترنت هدر می‌رفت.

دو افزوده این مشکل را حل می‌کنند:

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

- `--ffmpeg-location` به کاربر اجازه می‌دهد بدون تغییر هیچ تنظیمی در سیستم، به یک FFmpeg سالم یا پوشهٔ آن اشاره کند.
- `check_ffmpeg` پیش از نخستین دانلود و فقط **یک بار** `ffmpeg -version` را اجرا می‌کند. فایل ناموجود، از کار افتادن یا گیر کردن (با `timeout` ۱۵ ثانیه‌ای) به یک `RuntimeError` همراه با مسیر، کد خروج، توصیه و پیام خود FFmpeg تبدیل می‌شود؛ `main` آن را در یک خط چاپ می‌کند و با کد خروج ۱ پایان می‌یابد.

این همان اصل **زود شکست بخور** (fail fast) است: چیزهایی را که به آن‌ها وابسته‌اید در ابتدا بررسی کنید و روشن گزارش دهید، به‌جای اینکه بعداً در جایی گیج‌کننده شکست بخورید. کد خروج به شکل شانزده‌شانزدهی نمایش داده می‌شود، چون ویندوز چنین خطاهایی را این‌گونه مستند می‌کند؛ عبارت `returncode & 0xffffffff` عدد منفی پایتون برای این کدها را به شکل آشنای بدون علامت تبدیل می‌کند.

سه آزمون این بررسی را بدون نیاز به یک FFmpeg خراب پوشش می‌دهند: نبود FFmpeg (چیزی برای بررسی نیست)، مسیری که وجود ندارد و یک نتیجهٔ ساختگی `subprocess.run` با کد خروج ۱.

۹.۷ درس ۸: توضیح کد باید راست بگوید

هنگام بازبینی این تغییرها، یک خط چنین بود:

```
"ignoreerrors": True,  # Stop on errors, including a failed merge.
```

کد درست بود، اما توضیحش برعکس می‌گفت: `True` یعنی **متوقف نشو**؛ ویدیوی ناموفق را گزارش کن، از آن بگذر و بقیهٔ پلی‌لیست را ادامه بده. توضیح اشتباه از نبود توضیح بدتر است، چون خواننده به آن اعتماد می‌کند. اکنون چنین است:

```
# Skip a video that fails (private, deleted, failed merge) and continue
# with the rest of the playlist instead of stopping.
"ignoreerrors": True,
```

سپس این رفتار به‌طور واقعی آزمون شد: فهرستی که با یک ویدیوی ناموجود شروع می‌شد، برای آن یک خط ERROR چاپ کرد، ویدیوی بعدی را باز هم دانلود کرد و با پیام «برخی فایل‌ها دانلود نشدند» تمام شد. هر بار که کدی را تغییر می‌دهید، توضیح‌های اطراف آن را هم دوباره بخوانید.

۱۰.۷ خلاصهٔ تغییرها

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

مشکل یا نیاز	تغییر و روش آزمون آن
قطع شبکه دانلود تصویر را متوقف کرد	`"retries": 10`؛ یک آزمون واحد و تکرار دانلود واقعی
zsh به پیوندها بک‌اسلش اضافه کرد	`url.replace(...)` در `main`؛ آزمون با یک پیوند واقعیِ تغییریافته در zsh
ویدیوها با کد قاطی می‌شدند	`DOWNLOAD_FOLDER` و `code/downloads/` در `.gitignore`؛ یک آزمون واحد و `git check-ignore`
فقط صدا	`--audio` و پس‌پردازندهٔ MP3؛ دو آزمون واحد و دانلود واقعی یک ویدیو و یک پلی‌لیست
Ctrl+C شبیه خرابی بود	`try/except KeyboardInterrupt`؛ آزمون با متوقف کردن یک دانلود واقعی
انتخاب کیفیت؛ ویدیوهای عمودی	`--quality`، `--exact-quality` و `res:` در `format_sort`؛ چهار آزمون واحد و دانلود واقعی ویدیوی افقی و عمودی
FFmpeg پیدا نشد یا خراب بود	`--ffmpeg-location` و `check_ffmpeg`؛ چهار آزمون واحد و یک برنامهٔ واقعاً خراب
یک توضیح اشتباه	اصلاح توضیح `ignoreerrors`؛ یک فهرست واقعی با ویدیوی ناموجود

ایستگاه بررسی

می‌توانید توضیح دهید چرا دانلود Full HD پیش از فایل نهایی `.mp4` دو فایل می‌سازد، چرا مقدار پیش‌فرض `retries` در پایتون با دستور ترمینال فرق دارد، چرا برنامه بک‌اسلش‌ها را از پیوند حذف می‌کند و چرا فیلتر ارتفاع برای ویدیوی عمودی کیفیت اشتباه را انتخاب می‌کند.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

فصل ۸

رفع اشکال و گام‌های بعدی

هدف فصل

مشکل‌های رایج را سریع برطرف کنید و یک بهبود کوچک و کاربردی را برای ساخت در گام بعد انتخاب کنید.

۱.۸ مشکل‌های رایج

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

پیام یا نشانه	راه‌حل
`ModuleNotFoundError`	محیط فعال نیست یا بسته‌ها نصب نشده‌اند. محیط را فعال کنید و دستور `python -m pip install -r requirements.txt` را اجرا کنید.
Sign in to confirm you’re not a bot	یوتیوب را در یک مرورگر باز کنید و وارد حساب خود شوید. مرورگر را ببندید و دستور را دوباره با `--browser firefox` (یا نام مرورگر خودتان) اجرا کنید. خاموش‌کردن VPN یا تغییر سرور آن هم می‌تواند کمک کند.
No supported JavaScript runtime	`deno` نصب نیست. محیط را فعال کنید یا `python -m pip install deno` را اجرا کنید.
Got error: ... bytes read, ... more expected و در پوشه یک فایل `.f137.mp4.part` و یک فایل `.f140.m4a` مانده است	هنگام دانلود بخش تصویر، شبکه قطع شده است. یوتیوب تصویر و صدا را در دو بخش جدا می‌فرستد و FFmpeg فقط وقتی هر دو کامل باشند آن‌ها را ترکیب می‌کند. برنامه خودش ۱۰ بار دوباره تلاش می‌کند؛ اگر باز هم شکست خورد، **همان دستور** را دوباره اجرا کنید. `yt-dlp` بخش نیمه‌کاره را ادامه می‌دهد، هر دو را ترکیب می‌کند و فایل‌های موقت را پاک می‌کند.
Could not copy Chrome cookie database	کروم باز است و کوکی‌هایش را قفل کرده است. همهٔ پنجره‌های کروم را ببندید و دوباره امتحان کنید یا مرورگر دیگری را به کار ببرید.
فایل صدا ندارد یا فقط کیفیت پایین ذخیره می‌شود	FFmpeg نصب نیست یا پیدا نمی‌شود. آن را نصب کنید و در یک ترمینال تازه `ffmpeg -version` را بررسی کنید.
قبلاً کار می‌کرد، اما حالا با HTTP Error 403 یا خطای مشابه شکست می‌خورد	یوتیوب چیزی را تغییر داده است. کتابخانه را به‌روز کنید: `python -m pip install -U "yt-dlp[default]"`.
Private video یا Video unavailable	این ویدیو بدون اجازه قابل تماشا نیست. ویدیوهای دیگر پلی‌لیست همچنان دانلود می‌شوند و برنامه با پیام «برخی فایل‌ها دانلود نشدند» تمام می‌شود.
Requested format is not available پس از `--exact-quality`	این کیفیت دقیق برای این ویدیو وجود ندارد؛ پس از آن گذشت. کیفیت دیگری با `--quality` انتخاب کنید یا `--exact-quality` را ننویسید تا نزدیک‌ترین کیفیت پذیرفته شود.
Error: Cannot run FFmpeg at ...	مسیری که با `--ffmpeg-location` داده‌اید اشتباه است. خود برنامهٔ FFmpeg یا پوشه‌ای را بدهید که آن را در بر دارد (معمولاً یک پوشهٔ `bin`).
Error: FFmpeg exists but cannot run ... (exit code 0x...)	این FFmpeg خراب است یا یک کتابخانه کم دارد (`0xC0000135` در ویندوز یعنی یک DLL پیدا نشد). FFmpeg را دوباره نصب کنید یا با `--ffmpeg-location` به یک نسخهٔ سالم اشاره کنید. انتهای پیام، توضیح خود FFmpeg است.
zsh: no matches found یا پیوند در نویسهٔ & بریده می‌شود	پیوند را داخل گیومه بنویسید.
به‌جای یک ویدیو، کل پلی‌لیست شروع می‌شود	پیوند بخش `&list=` دارد. آن بخش را حذف کنید و دوباره اجرا کنید.

وقتی کمک می‌خواهید (مثلاً در یک Issue در گیت‌هاب)، سیستم‌عامل، نسخهٔ پایتون، دستور دقیق و پیام

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

کامل خطا را بنویسید. پیش از آن، هر اطلاعات خصوصی را حذف کنید.

۲.۸ تمیز نگه‌داشتن پروژه در گیت

فایل `.gitignore` پروژه به گیت می‌گوید کدام فایل‌ها را هرگز ذخیره نکند. این فایل کل پوشهٔ `code/downloads/`، فایل‌های رسانه‌ای در هر جای پروژه (مانند `*.mp4`، `*.mp3` و `*.webm`)، فایل‌های نیمه‌کارهٔ `*.part` و پوشهٔ `.venv` را فهرست کرده است. پیش از هر کامیت دستور `git status` را اجرا کنید: هیچ فایل ویدیویی نباید در فهرست باشد.

۳.۸ ایده‌هایی برای تمرین

هر ایده فقط چند خط کد لازم دارد و یک مهارت تازه را تمرین می‌دهد. برای هر تغییر یک آزمون هم اضافه کنید.

۱. **انتخاب قالب صدا.** گزینهٔ `--audio-format` را با `choices=["mp3", "m4a", "opus"]` اضافه کنید و مقدار آن را به‌جای `"mp3"` ثابت، در `preferredcodec` به کار ببرید:

```
parser.add_argument("--audio-format", choices=["mp3", "m4a", "opus"],
                    default="mp3", help="file type for --audio")
```

۲. **نمایش کیفیت‌های موجود.** گزینهٔ `--list` را اضافه کنید که مانند `yt-dlp -F` فقط قالب‌های یک ویدیو را چاپ کند تا کاربر بتواند `--quality` موجودی را انتخاب کند:

```
options["listformats"] = True   # print the table, download nothing
```

۳. **چند پیوند با هم.** `url` را به `nargs="*"` تغییر دهید و کل لیست را به `ydl.download` بدهید.

۴. **زیرنویس.** زیرنویس انگلیسی و فارسی را کنار هر ویدیو ذخیره کنید:

```
options["writesubtitles"] = True
options["subtitleslangs"] = ["en", "fa"]
```

۵. **خواندن پیوندها از فایل.** از فایل `links.txt` در هر خط یک پیوند بخوانید:

```
links = Path("links.txt").read_text().splitlines()
```

همهٔ گزینه‌های موجود در راهنمای yt-dlp توضیح داده شده‌اند. گزینه‌ای که در خط فرمان نوشته می‌شود، مانند `--write-subs`، در پایتون به کلیدی در دیکشنری تبدیل می‌شود، مانند `writesubtitles`.

ایستگاه بررسی

می‌توانید علت یک دانلود ناموفق را از پیامش پیدا کنید، `yt-dlp` را به‌روز کنید و خودتان گزینهٔ تازه‌ای به دیکشنری تنظیم‌ها اضافه کنید.

تبریک! شما یک ابزار کامل و آزمون‌شدهٔ پایتون را از یک پوشهٔ خالی ساختید.

صفحه‌های اجتماعی Alireza Khajehvandi
LinkedIn YouTube Telegram Instagram GitHub Portfolio

دربارهٔ نویسنده

ایده و محتوای این آموزش و پروژه را Alireza Khajehvandi ساخته است تا تازه‌کارها در زمانی کوتاه ابزاری کاربردی بسازند و هر خطی را که اجرا می‌کنند، بفهمند.

برای ادامهٔ یادگیری و ارتباط:

- LinkedIn
- کانال YouTube
- کانال تلگرام
- Instagram
- GitHub
- وب‌سایت شخصی و نمونه‌کارها